semicolon

FASTAPI · LAYERED ARCHITECTURE · MONGODB · PYTEST

Building a Todo API with Layered Architecture in FastAPI

The complete student guide: every layer, every class and every decision explained — Model → Repository → Service → Controller, from a plain Python list to MongoDB, with tests.

WRITTEN BY

Sikiru Ayinlade

Software Engineer / Facilitator

Contents

About This Book . . . 4

Who this book is for . . . 4

What you will build . . . 4

How the callout boxes work . . . 4

About the screenshots . . . 5

1. What Is Layered Architecture? . . . 6

1.1 The problem: everything in one place . . . 6

1.2 The idea: one job per layer . . . 6

1.3 The dependency rule . . . 7

1.4 Following one request through the layers . . . 8

1.5 A few words you will meet in this book . . . 8

2. Project Setup . . . 9

2.1 The tools . . . 9

2.2 Create the project and install the libraries . . . 9

2.3 Create the folder structure . . . 10

3. Step 1 — Build the Create Todo Endpoint . . . 13

3.1 Model layer . . . 13

3.2 Repository layer (List) . . . 18

3.3 Service layer . . . 23

3.4 Controller layer (routes / urls) . . . 25

3.5 Wire the layers together . . . 27

3.6 Run it and test the Create endpoint . . . 29

4. Step 2 — Complete the Full CRUD . . . 33

4.1 Add these methods to the Service . . . 33

4.2 Add these routes to the Controller . . . 35

4.3 Exercise all five endpoints . . . 39

5. Step 3 — Swap the List for MongoDB (pymongo) . . . 40

5.1 Install pymongo and start MongoDB . . . 40

5.2 Database connection . . . 41
5.3 MongoDB repository . . . 42
5.4 The one-line swap . . . 45
5.5 Final project structure . . . 47

6. Step 4 — Test Your Endpoints . . . 49
6.1 Install test dependencies . . . 49
6.2 Unit test the Service . . . 50
6.3 Integration test the Controller . . . 52
6.4 Run the tests . . . 55

7. Putting It All Together . . . 57
7.1 One request, every layer . . . 57
7.2 Where does this code belong? . . . 57
7.3 The layering checklist . . . 58
7.4 The patterns you used . . . 58
7.5 Mistakes to avoid . . . 58

8. Troubleshooting . . . 59

9. Exercises and Self-Check . . . 60
9.1 Practice exercises . . . 60
9.2 Self-check questions . . . 60
9.3 Be ready to defend your design . . . 61

10. Glossary . . . 62

Closing Note . . . 64

About This Book

This book teaches you **layered architecture** by making you build something real: a Todo API with FastAPI. Every section follows the same rhythm: first the **idea** (what this piece is and why it exists), then the **code**, then a **walkthrough** that explains the code piece by piece, and finally a **screenshot** of the same code in an IDE so you can check your own work against it.

You are not meant to copy the code blindly. The goal is that, by the last page, you can explain to someone else why each file exists, what it is allowed to know, and what you would change (and what you would leave alone) if the requirements changed. That is what separates a person who has followed a tutorial from an engineer.

Who this book is for

- You know basic Python: variables, functions, classes, `import`, and `if` / `for`.
- You have seen type hints such as `name: str`. If not, section 3.1 explains the ones we use.
- You are comfortable opening a terminal and running a command.
- You do **not** need to know FastAPI, Pydantic, MongoDB or pytest. We introduce each one when we first need it.

What you will build

Step	What you add	What you learn
Step 1	The **Create Todo** endpoint, end to end, using a plain Python list for storage.	Models and DTOs, the repository contract, the service, the controller, dependency injection.
Step 2	The remaining endpoints: **list, get one, update, delete**.	Business rules, domain exceptions, HTTP status codes, partial updates.
Step 3	A MongoDB repository, swapped in by changing **one line**.	Why the architecture works: the same app on a different database.
Step 4	**Tests** for the service and for the live endpoints.	Unit vs integration tests, fixtures, and overriding dependencies.

How the callout boxes work

> KEY IDEA
> A **Key idea** box holds the one thing you should be able to say in your own words after reading the section.

> WATCH OUT
> A **Watch out** box warns about a mistake that students make often.

> **TRY IT**
> A **Try it** box gives you something small to run yourself. Do these. Reading about code is not the same as running it.

> **NOTE**
> A **Note** box adds a detail that is useful but not essential on a first read.

About the screenshots

The screenshots in this book were taken in PyCharm on macOS with a dark theme. Your editor, theme and window layout may look different. That is fine. What matters is the **file names**, the **folder structure** and the **code**. Where a screenshot is scrolled so that a few lines are hidden, the text next to it lists the complete file.

Every code listing in the book is complete and can be copied into your project. If a listing differs from a screenshot, the explanation next to it tells you why.

1. What Is Layered Architecture?

1.1 The problem: everything in one place

Imagine you build your first API in the quickest way possible. Everything lives inside the route function: checking the input, deciding what is allowed, and storing the data.

main.py (the "all in one" version. Do NOT build this)

```
from fastapi import FastAPI, HTTPException

app = FastAPI()
todos = []                      # storage, right here in the route file


@app.post("/todos")
def create_todo(body: dict):
    if "title" not in body or not body["title"].strip():     # validation
        raise HTTPException(status_code=422, detail="title is required")
    if len(todos) >= 100:                                    # business rule
        raise HTTPException(status_code=400, detail="too many todos")
    todo = {"id": len(todos) + 1, "title": body["title"], "completed": False}
    todos.append(todo)                                       # storage
    return todo
```

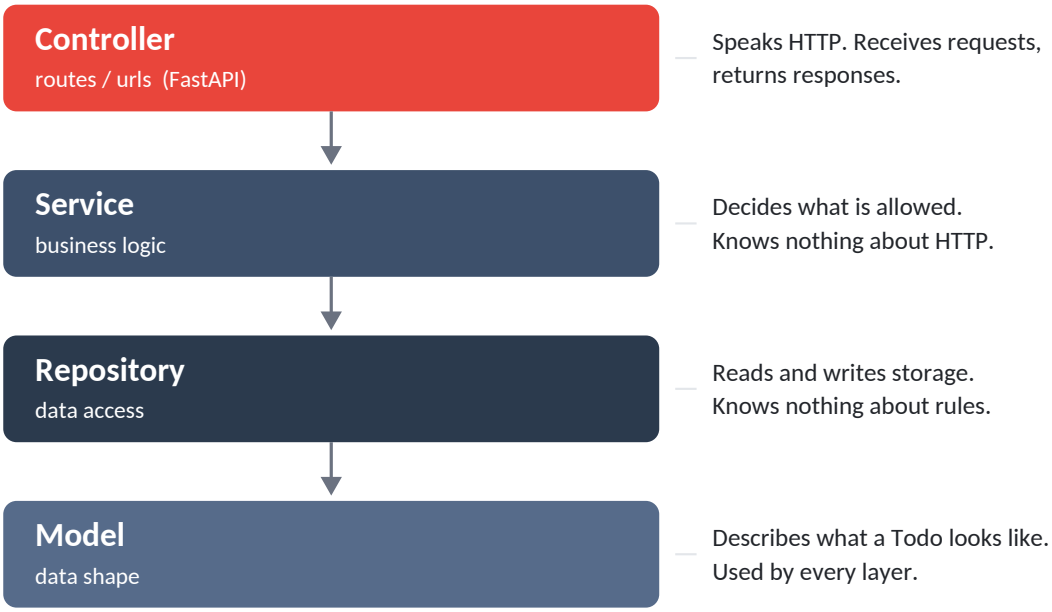
This works for a demo. It stops working the moment the app grows, because **one function now has four jobs**: it speaks HTTP, validates input, enforces rules and talks to storage. Three problems follow:

- **Changing the database means rewriting every route.** If `todos.append(...)` becomes a MongoDB call, every endpoint that touches `todos` must be edited.
- **You cannot test the rules without HTTP.** To check "no more than 100 todos" you have to send real requests.
- **Rules get duplicated.** The same rule copied into five routes will eventually be changed in four of them.

1.2 The idea: one job per layer

Layered architecture splits an application into independent layers, each with **one job**. A change in one layer (for example, swapping storage from a Python list to MongoDB) should not force changes in the others. The programming principle behind this has a name: **separation of concerns**. A closely related one is the **single responsibility principle**: a class or module should have one reason to change.

In this book we use four layers:



Layer	Folder	Its one job	It must NOT
Model	`models/`	Define the shape of a Todo, and of the data needed to create or update one.	Talk to a database or to HTTP.
Repository	`repositories/`	The only layer that knows how data is actually stored. Today a Python list, later MongoDB.	Contain business rules or know about HTTP.
Service	`services/`	Business logic. It decides things like "you cannot fetch a Todo that does not exist". It talks to the Repository, never to the database directly.	Know about HTTP, status codes or JSON.
Controller	`controllers/`	The FastAPI routes (your urls). Receives HTTP requests, calls the Service, returns HTTP responses.	Contain business logic or touch storage.

1.3 The dependency rule

Look at the diagram again. The arrows point **downwards**: the Controller calls the Service, the Service calls the Repository. Nothing ever calls **upwards**. The Repository has never heard of the Service, and the Service has never heard of the Controller.

There is one more subtlety that makes the whole design work. The Service does not depend on a **specific** repository (such as "the list one" or "the MongoDB one"). It depends on an **abstract contract** that says "a repository can add, get, list, update and delete todos". Any class that keeps that promise can be plugged in. This idea is called **dependency inversion**, and you will see it in action in sections 3.2 and 3.5.

> **KEY IDEA**
> Each layer may only talk to the layer directly below it, and only through a simple, well-defined interface. If you ever need to import something from a higher layer, a responsibility is in the wrong place.

1.4 Following one request through the layers

Here is what happens when a client sends `POST /todos/` with `{"title": "Buy milk"}`. You do not need to understand every term yet. Come back to this table after Step 1 and every row will make sense.

#	Where	What happens
1	Client	Sends an HTTP request: method `POST`, path `/todos/`, JSON body.
2	FastAPI	Matches the path to the route in the **Controller**. Parses the JSON into a `TodoCreate` object and validates it. Bad input is rejected here with a 422.
3	FastAPI	Builds the objects the route asks for: a `TodoRepository`, then a `TodoService` that receives it (dependency injection).
4	Controller	Calls `service.create_todo(payload)`. It adds no logic of its own.
5	Service	Turns the `TodoCreate` into a full `Todo` (new id, timestamps, `completed=False`) and asks the repository to save it.
6	Repository	Stores the todo (appends to a list now; inserts a document in MongoDB later) and hands it back.
7	Controller	Returns the `Todo`. FastAPI converts it to JSON and sends `201 Created`.

1.5 A few words you will meet in this book

Word	Short meaning (the glossary at the end has more)
Entity / domain model	The core object of your app. Here: `Todo`.
DTO (Data Transfer Object)	A simple object whose only job is to carry data between two parts of a system, for example from the client into your API.
Contract / interface	A list of methods a class promises to provide, without saying how. Here: `TodoRepository`.
Dependency injection	A class receives the things it needs instead of creating them itself.
Composition root	The one place where the real objects are chosen and connected. Here: `dependencies.py`.

> **WHY IT MATTERS**
> By the end of this guide you will change ONE line of code to move the entire app from an in-memory list to a real MongoDB database. If more than one file needs to change for that swap, a layer leaked a detail it should not have.

2. Project Setup

Before writing any code we need a project folder, an isolated Python environment, and the libraries we will use.

2.1 The tools

Tool	What it is	Why we use it
Python 3.10+	The language. The screenshots use a recent version of Python 3.	FastAPI and Pydantic need a modern Python.
uv	A fast Python package and project manager.	Creates the project, the virtual environment, installs libraries and runs commands, all with one tool.
An IDE	PyCharm in the screenshots; VS Code works equally well.	Autocomplete and error highlighting help you catch typos early.
MongoDB (Step 3)	A document database.	Our real database at the end of the book.

If you do not have `uv` yet, install it once (see the official installation page at docs.astral.sh/uv), for example:

Terminal

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

# or, if you already have pip
pip install uv
```

2.2 Create the project and install the libraries

Terminal

```
uv init todo_api --app
cd todo_api
uv add "fastapi[standard]" pydantic
```

Command	What it does
`uv init todo_api --app`	Creates a new folder `todo_api/` set up as an **application** (as opposed to a library). It adds a starter `main.py`, a `pyproject.toml`, a `.python-version` file and a `.gitignore`.
`cd todo_api`	Moves into the project folder. **Every command in this book is run from here.**
`uv add "fastapi[standard]" pydantic`	Installs the libraries and records them in `pyproject.toml`. The quotes stop your shell from treating the square brackets specially.

What did we just install?

Library	Role in our project
FastAPI	The web framework. It gives us routing (`@router.post(...)`), automatic request validation, dependency injection (`Depends`) and the interactive `/docs` page.
Pydantic	The data-validation library that FastAPI is built on. It gives us `BaseModel` and `Field`, which we use for every model and DTO. FastAPI already depends on it, but we import it directly, so we list it explicitly.
Uvicorn	The server that actually listens on a network port and passes HTTP requests to FastAPI. It comes bundled with the `standard` extra, so there is no separate install.

> NOTE
> `uv add "fastapi[standard]"` pulls in `uvicorn[standard]` automatically as part of the extra, so there is no separate uvicorn install. `uv init` also drops in a starter `main.py`, `pyproject.toml`, `.python-version` and a `.gitignore`. We will overwrite `main.py` with our own version in section 3.5.

The files uv created for you

File / folder	Purpose
`pyproject.toml`	The project's settings and list of dependencies. You will see `fastapi` and `pydantic` listed here.
`uv.lock`	The exact versions that were installed, so everyone on the team gets identical results. Do not edit by hand.
`.venv/`	The **virtual environment**: a private copy of Python and its libraries just for this project, so projects never interfere with each other.
`.python-version`	Tells uv which Python version this project uses.

> NOTE
> No `venv/` folder to create or activate. uv creates a `.venv/` automatically the first time you `uv add` or `uv run` something, and every command below runs inside it via `uv run`.

2.3 Create the folder structure

Now create one folder per layer inside `todo_api/`. (uv has already created `main.py` and `pyproject.toml` for you.)

Terminal (macOS / Linux)

```
mkdir controllers models repositories services
touch controllers/__init__.py models/__init__.py repositories/__init__.py services/__init__.py
touch controllers/todo_controller.py models/todo.py
touch repositories/todo_repository.py repositories/in_memory_todo_repository.py
touch services/todo_service.py dependencies.py
```

On Windows, simply create the same folders and files from your IDE (right-click the project, then **New**).

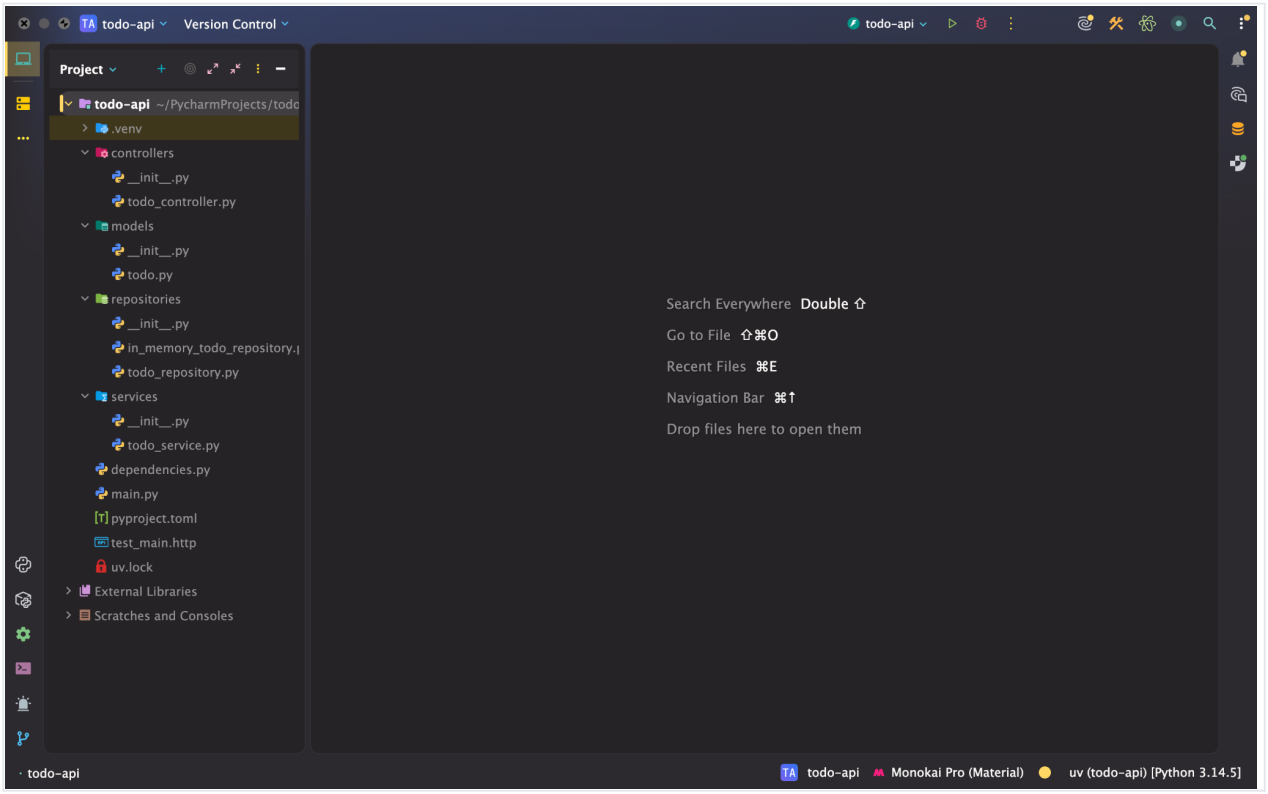


Figure 2.1: The project tree. One folder per layer, plus dependencies.py and main.py at the top.

Path	Layer	Will contain
`models/todo.py`	Model	`Todo`, `TodoCreate`, `TodoUpdate`
`repositories/todo_repository.py`	Repository	The abstract contract `TodoRepository`
`repositories/in_memory_todo_repository.py`	Repository	The list-backed implementation
`services/todo_service.py`	Service	`TodoService` and `TodoNotFoundError`
`controllers/todo_controller.py`	Controller	The FastAPI router with all the routes
`dependencies.py`	Wiring	Decides which repository is used and builds the service
`main.py`	Entry point	Creates the FastAPI app and attaches the router

Why every folder has an `__init__.py`

An empty file called `__init__.py` tells Python that a folder is a **package**, which makes imports such as `from models.todo import Todo` work. The import path starts at the project root (`todo_api/`), which is also why we always run our commands from that folder.

> **KEY IDEA**
> The folder names are the architecture. Anyone who opens the project can see at a glance where a piece of code belongs and, just as important, where it does **not** belong.

3. Step 1 — Build the Create Todo Endpoint

We build ONE feature end to end, creating a todo, before touching anything else. This is called a **vertical slice**: a thin cut through every layer that works on its own. In order: Model, then Repository, then Service, then Controller, then the wiring.

Why in that order? Because of the dependency rule from chapter 1. The Model depends on nothing, the Repository depends on the Model, the Service depends on the Repository and the Model, and the Controller depends on the Service. If you build from the bottom up, every file you write can already import everything it needs.

Section	File you create	What goes in it
3.1	`models/todo.py`	The three Pydantic classes
3.2	`repositories/todo_repository.py`	The abstract contract
3.2	`repositories/in_memory_todo_repository.py`	The list-backed storage
3.3	`services/todo_service.py`	Business logic for creating a todo
3.4	`controllers/todo_controller.py`	The `POST /todos/` route
3.5	`dependencies.py` and `main.py`	Connecting everything and starting the app

3.1 Model layer

The Model layer defines what a Todo looks like, and what data is required to create or update one. We use **three Pydantic classes for three different jobs**. Before reading the code, let us understand the tools it uses.

What is a DTO?

A **DTO** (Data Transfer Object) is a plain object whose only job is to **carry data** from one place to another: for example, from the client into your API, or from your API back out to the client. A DTO has fields, and maybe validation rules, but no behaviour and no knowledge of databases.

In our project, `TodoCreate` and `TodoUpdate` are **input DTOs**: they describe exactly what the client is allowed to send. `Todo` is the full model: what we store and what we return.

Why not use one `Todo` class for everything? Because the three situations need different rules:

Class	Used for	Who fills in the values	Fields
`TodoCreate`	The body of `POST /todos/`	The **client**	`title` (required), `description` (optional)
`TodoUpdate`	The body of `PATCH /todos/{id}`	The **client**	`title`, `description`, `completed`: all optional, so the client can change just one

Class	Used for	Who fills in the values	Fields
`Todo`	What we store and what we return	The **server** (generated and defaulted)	`id`, `title`, `description`, `completed`, `created_at`, `updated_at`

> **WHY IT MATTERS**
> If a single `Todo` class were used for input, a client could send their own `id`, mark a todo as `completed` at creation, or back-date `created_at`. Separate input DTOs make it **impossible** for the client to set fields that only the server should control. They also let your API contract change without touching your storage model, and the other way round.

What is Pydantic's BaseModel?

`BaseModel` is the class that all Pydantic models inherit from. When you write a class that extends it, you list the fields with type hints, and Pydantic gives you a lot for free:

What you get	What it means
Validation	When you create an object, Pydantic checks every field against its type and rules. Invalid data raises a `ValidationError`.
Type conversion	Compatible values are converted. The string `"5"` becomes the integer `5` for an `int` field; a UUID string becomes a real `UUID`.
Defaults	Fields you leave out get their default value, or one produced by a function (`default_factory`).
Serialization	`model_dump()` turns the object into a plain dictionary; `model_dump_json()` into a JSON string.
A JSON schema	FastAPI reads your models to build the interactive documentation at `/docs` automatically.

You can see this for yourself in a Python shell once the file below exists. Run `uv run python` from the project folder:

Python shell

```
>>> from models.todo import TodoCreate
>>> TodoCreate(title="Buy milk")
TodoCreate(title='Buy milk', description=None)
>>> TodoCreate(title="")
pydantic_core._pydantic_core.ValidationError: 1 validation error for TodoCreate
title
  String should have at least 1 character [type=string_too_short, input_value='', input_type=str]
```

That error message is exactly what FastAPI turns into an HTTP **422** response when a client sends bad data. We never wrote an `if` statement to check the title. The model itself refuses to exist in an invalid state.

Field(): rules and defaults for one field

Writing `title: str` says "title is a string". `Field(...)` lets you add rules and defaults on top of the type:

In the code	Meaning
`Field(..., min_length=1, max_length=200)`	The first argument is the default. `...` (called Ellipsis) means **no default: this field is required**. The title must be between 1 and 200 characters long, so an empty title is rejected.
`Field(default_factory=uuid4)`	When no value is given, **call** `uuid4()` to create one. Every new `Todo` gets its own fresh id.
`Field(default_factory=lambda: datetime.now(timezone.utc))`	Same idea for the timestamps: call the small function each time a `Todo` is created.

Optional, None and "required or not"

`Optional[str]` means "a string **or** `None`". Writing `description: Optional[str] = None` means: the field may be a string, may be `None`, and if the client leaves it out it defaults to `None`. (In Python 3.10+ you may also write `str | None`.)

Declaration	Required?	Can it be None?
`title: str`	Yes	No
`description: Optional[str] = None`	No	Yes
`description: Optional[str]` (no default)	**Yes** (in Pydantic v2)	Yes
`completed: bool = False`	No	No

> **WATCH OUT**
> In Pydantic v2, `Optional[str]` on its own does **not** make a field optional. It only allows `None` as a value. To make the field optional you must also give it a default such as `= None`. This is why every field of `TodoUpdate` ends with `= None`.

UUID and uuid4

A `UUID` is a 128-bit identifier written like `7c9e6679-7425-40de-944b-e07fc1f90ae7`. `uuid4()` generates a random one. We use UUIDs instead of 1, 2, 3 because (a) the application can create the id **before** saving, so it does not depend on any database, (b) ids cannot be guessed by counting, and (c) they work the same in a list, in MongoDB and in SQL.

datetime and timezone.utc

`datetime.now(timezone.utc)` returns the current time **with a time-zone attached** (UTC). Storing times in UTC is the safest habit: servers and users live in different time zones, and UTC avoids confusion. (Older code uses `datetime.utcnow()`. It is deprecated in recent Python versions, so avoid it.)

default_factory and why we write lambda

A normal default is evaluated **once**, when Python reads the class. A `default_factory` is a function that is called **every time** an object is created.

Why default_factory matters

```
# WRONG: datetime.now() runs once, when the module is imported.
# Every todo would share the same "created_at": the moment the server started.
created_at: datetime = datetime.now(timezone.utc)

# RIGHT: the lambda runs each time a Todo is created.
created_at: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))
```

`uuid4` can be passed directly because it is already a function that needs no arguments. `datetime.now(timezone.utc)` needs an argument, so we wrap it in a tiny `lambda:` to make a function that needs none.

The code: models/todo.py

models/todo.py

```
from pydantic import BaseModel, Field
from typing import Optional
from uuid import UUID, uuid4
from datetime import datetime, timezone


class TodoCreate(BaseModel):
    """What the client sends us to create a todo, also known as DTO"""
    title: str = Field(..., min_length=1, max_length=200)
    description: Optional[str] = None


class TodoUpdate(BaseModel):
    """What the client sends us to update a todo (all optional). update DTO"""
    title: Optional[str] = None
    description: Optional[str] = None
    completed: Optional[bool] = None


class Todo(BaseModel):
    """The full todo, as stored and as returned to the client."""
    id: UUID = Field(default_factory=uuid4)
    title: str
    description: Optional[str] = None
    completed: bool = False
    created_at: datetime = Field(
        default_factory=lambda: datetime.now(timezone.utc)
    )
    updated_at: datetime = Field(
        default_factory=lambda: datetime.now(timezone.utc)
    )
```

WATCH OUT

In the screenshot below the editor is scrolled by one line, so the very first line is hidden. Do not forget line 1: `from pydantic import BaseModel, Field`.

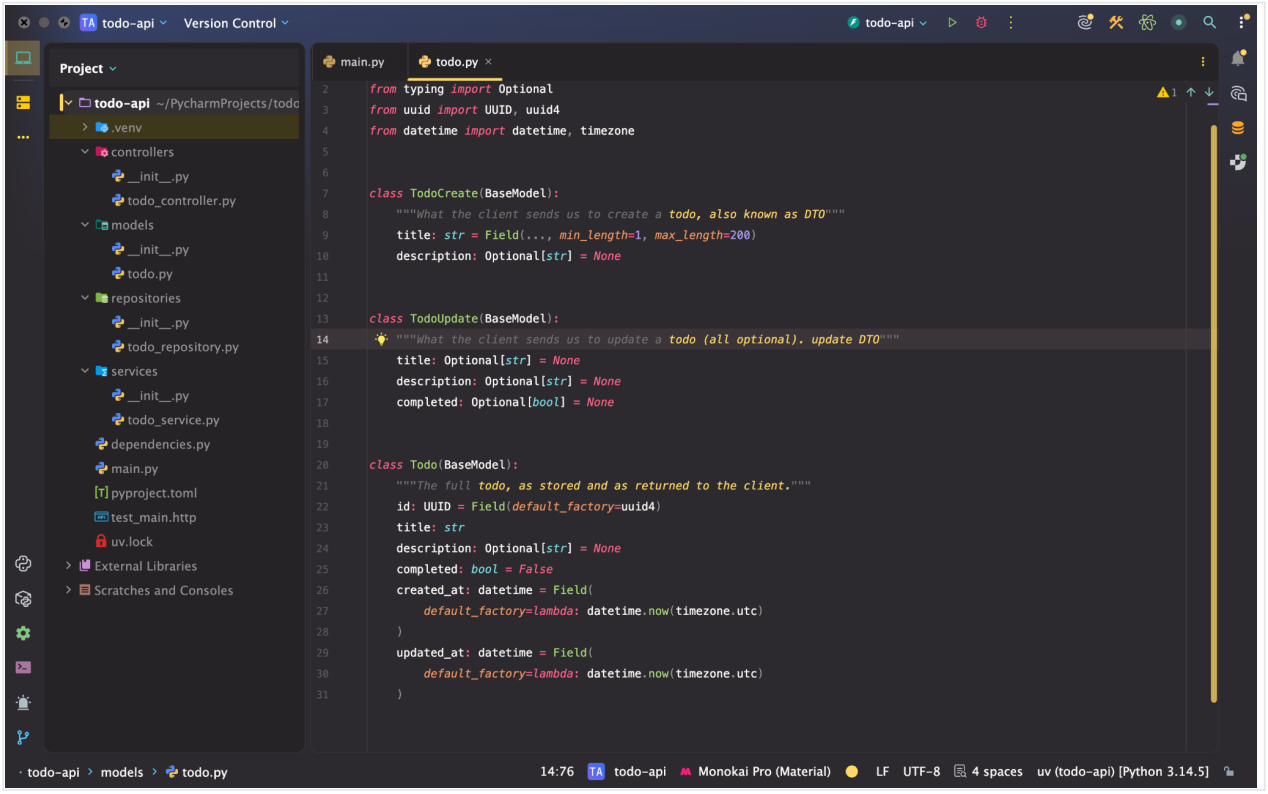


Figure 3.1: models/todo.py in the IDE.

Reading the code, class by class

Code	What it says
`class TodoCreate(BaseModel):`	Defines a Pydantic model named `TodoCreate`. The text in triple quotes is a **docstring**: a description that also appears in the generated API documentation.
`title: str = Field(..., min_length=1, max_length=200)`	A required string, 1 to 200 characters. A missing or empty title is rejected before any of our code runs.
`description: Optional[str] = None`	An optional note. Leave it out and it is `None`.
`class TodoUpdate(BaseModel):`	Every field is optional. A client who only wants to tick a todo sends `{"completed": true}` and nothing else.
`class Todo(BaseModel):`	The complete record. `id`, `completed`, `created_at` and `updated_at` all have defaults, so the Service only has to supply `title` and `description`.
`completed: bool = False`	Every new todo starts as not completed.

TRY IT

Open `uv run python` and create a `Todo(title="Test")`. Print it. Notice that you get an `id`, two timestamps and `completed=False` without writing any of them. Then try `Todo()` with no title and read the error.

Common mistakes in this layer

- Forgetting `= None` on an optional field (see the warning above).
- Putting `datetime.now()` directly as a default instead of using `default_factory`.
- Adding database code or HTTP code to a model. Models only describe data.
- Reusing `Todo` as the request body. Always use a DTO for input.

3.2 Repository layer (List)

The repository is the ONLY place that touches storage. A **repository** behaves like a collection of todos: you can add one, get one, list them all, update one, delete one. The rest of the app does not know or care how those operations are carried out.

We write an **abstract base class** first (the contract), then an **in-memory implementation** backed by a plain Python list. Every method the Service will ever need is declared here.

ABC and @abstractmethod: writing a contract

`ABC` stands for **Abstract Base Class**. A class that inherits from `ABC` and has methods marked `@abstractmethod` is a contract: "anyone who wants to be a `TodoRepository` **must** provide these methods". Python enforces it. You cannot create an object from the contract itself, and you cannot create an object from a subclass that forgot a method.

Python shell

```
>>> from repositories.todo_repository import TodoRepository
>>> TodoRepository()
TypeError: Can't instantiate abstract class TodoRepository without an implementation
for abstract methods 'add', 'delete', 'get', 'list', 'update'
```

(The exact wording of the message varies slightly between Python versions.) The `...` inside each method is Python's **Ellipsis**. It is simply a placeholder body that means "no code here", like `pass`.

The five methods of the contract

Method	Purpose	Returns	If the todo does not exist
`add(todo)`	Save a new todo	The saved `Todo`	n/a
`get(todo_id)`	Fetch one todo by id	`Optional[Todo]`	Returns `None`
`list()`	Fetch all todos	`List[Todo]`	Returns an empty list
`update(todo_id, data)`	Change some fields of a todo	`Optional[Todo]` (the updated one)	Returns `None`
`delete(todo_id)`	Remove a todo	`bool`	Returns `False`

Three design decisions are hiding in this table:

- **Returning `None` instead of raising an error.** The repository only reports a fact ("nothing found"). Whether a missing todo is a problem is a **business decision**, so it belongs to the Service.
- **`update` takes a plain `dict`, not a `TodoUpdate`.** This keeps the repository independent of the API's DTOs. It receives only "which fields to change".

- **We import `List` from `typing`, not the built-in `list[...]`.** Inside the class we define a method called `list`. After that line, the name `list` inside the class means the method, not the built-in type, so `list[Todo]` would break. `List[Todo]` avoids the clash.

Create the contract:

repositories/todo_repository.py

```python
from abc import ABC, abstractmethod
from typing import List, Optional
from uuid import UUID
from models.todo import Todo


class TodoRepository(ABC):
    """Contract every repository (list, MongoDB, ...) must follow."""

    @abstractmethod
    def add(self, todo: Todo) -> Todo:
        ...

    @abstractmethod
    def get(self, todo_id: UUID) -> Optional[Todo]:
        ...

    @abstractmethod
    def list(self) -> List[Todo]:
        ...

    @abstractmethod
    def update(self, todo_id: UUID, data: dict) -> Optional[Todo]:
        ...

    @abstractmethod
    def delete(self, todo_id: UUID) -> bool:
        ...
```

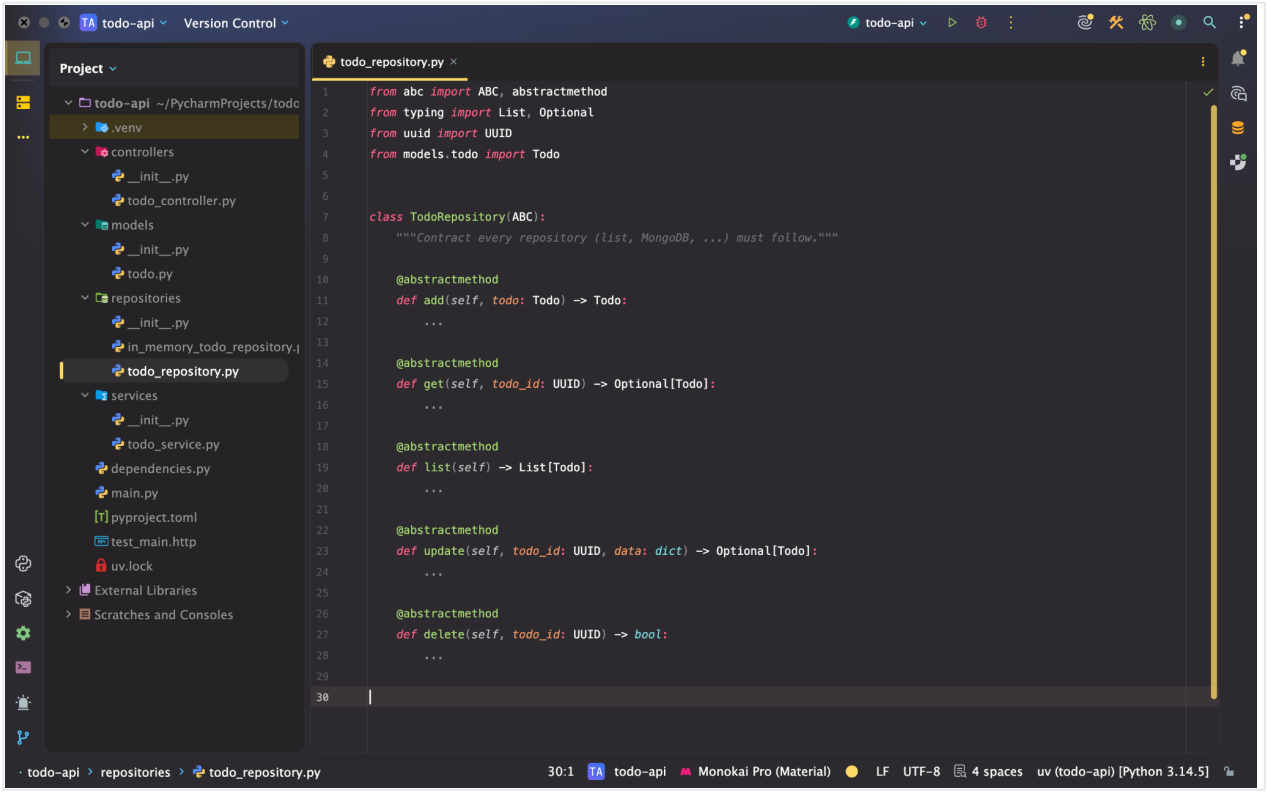


Figure 3.2: repositories/todo_repository.py. The contract has signatures only, with no storage code.

> **KEY IDEA**
> The contract says **what** a repository can do. An implementation says **how**. The Service will only ever see the contract.

The in-memory implementation

Now, in its own file, the implementation that keeps todos in a plain Python list. Note the file name: `in_memory_todo_repository.py`.

repositories/in_memory_todo_repository.py

```python
from typing import List, Optional
from uuid import UUID
from models.todo import Todo
from repositories.todo_repository import TodoRepository


class InMemoryTodoRepository(TodoRepository):
    """Stores todos in a plain Python list. No database needed yet."""

    def __init__(self):
        self._todos: List[Todo] = []

    def add(self, todo: Todo) -> Todo:
        self._todos.append(todo)
        return todo

    def get(self, todo_id: UUID) -> Optional[Todo]:
        for todo in self._todos:
            if todo.id == todo_id:
                return todo
```

```
        return None

    def list(self) -> List[Todo]:
        return self._todos

    def update(self, todo_id: UUID, data: dict) -> Optional[Todo]:
        todo = self.get(todo_id)
        if todo is None:
            return None
        updated = todo.model_copy(update=data)
        index = self._todos.index(todo)
        self._todos[index] = updated
        return updated

    def delete(self, todo_id: UUID) -> bool:
        todo = self.get(todo_id)
        if todo is None:
            return False
        self._todos.remove(todo)
        return True
```

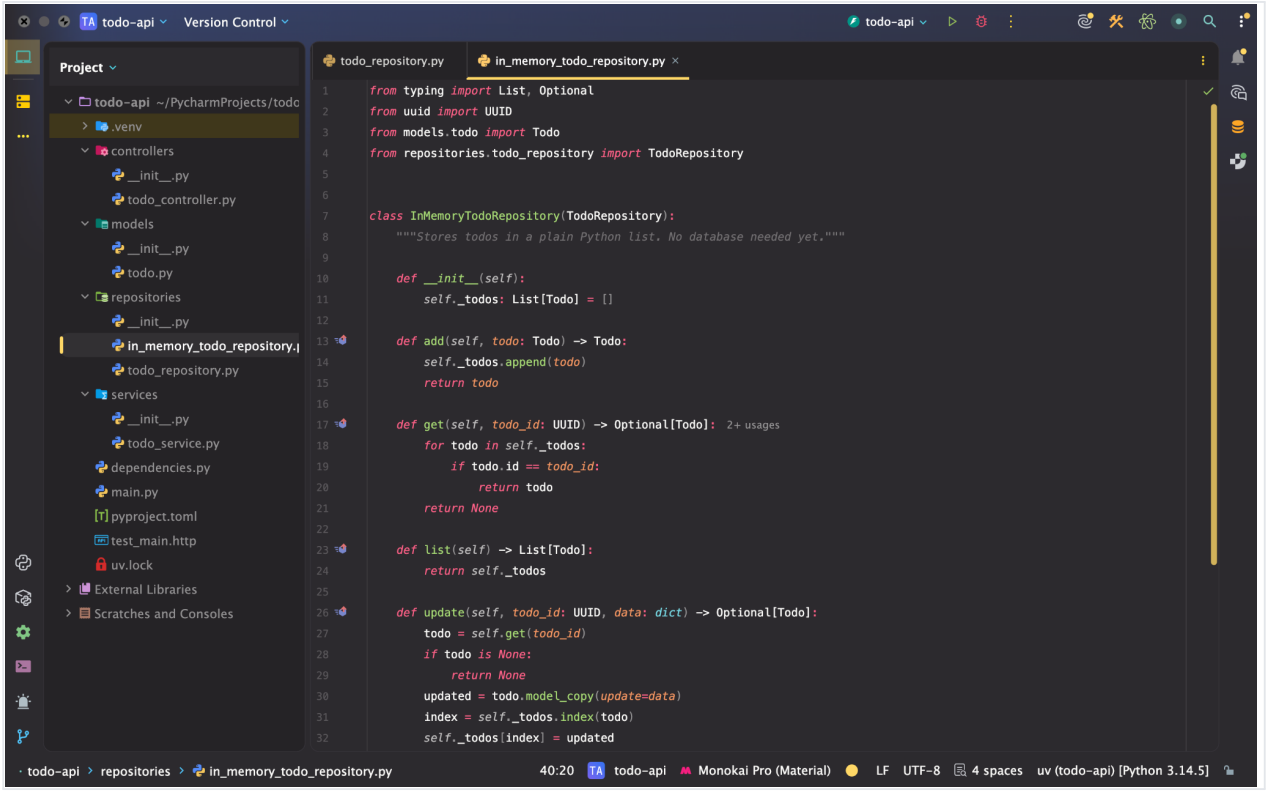


Figure 3.3: in_memory_todo_repository.py (top of the file).

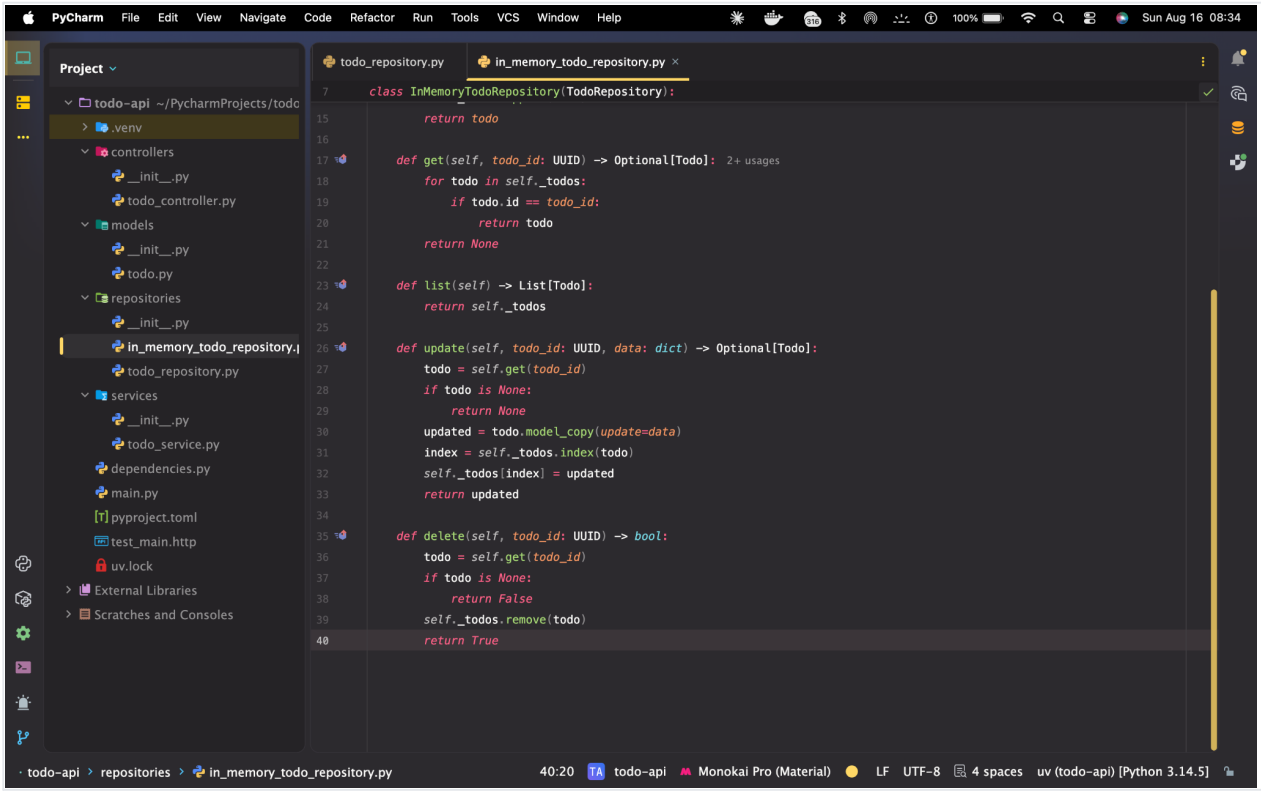


Figure 3.4: in_memory_todo_repository.py (scrolled down to update and delete).

Reading the code

Code	What it does
`class InMemoryTodoRepository(TodoRepository):`	Says "I keep the promise made by `TodoRepository`". If one of the five methods were missing, Python would refuse to create the object.
`self._todos: List[Todo] = []`	The storage: an empty list owned by **this object**. The leading underscore is a convention that means "internal, do not touch from outside".
`add`	Appends the todo to the list and returns it.
`get`	Walks through the list and returns the todo whose `id` matches, or `None` if the loop ends without a match.
`list`	Returns the whole list.
`update`	Finds the todo (or returns `None`). `model_copy(update=data)` makes a **copy** of the todo with the given fields replaced. We then put the copy in the same position in the list.
`delete`	Finds the todo; returns `False` if it is not there, otherwise removes it and returns `True`.

> **NOTE**
> `model_copy(update=...)` does not run validation on the new values. That is fine here, because the data was already validated when it entered through `TodoUpdate`. Also, looking a todo up by walking a list is slow for large lists. A dictionary keyed by id would be faster. For learning, the simple loop is easier to read.

> **NOTE**
> We declared all five methods on the abstract class up front (it costs nothing), but the Service and Controller will only call `add()` in this step. The rest come in Step 2.

3.3 Service layer

The Service holds the **business logic**: the rules of your application that are independent of both HTTP and storage. Typical examples are "a title must be unique", "a user cannot have more than 50 open todos", or "you cannot fetch a todo that does not exist".

For now, the service has one job: create a todo through the repository. It is thin today, and that is fine. The layer exists so that, when the first rule arrives, it already has a home.

Three ideas in this file

1. A custom exception. `TodoNotFoundError` is our own error type. Its name says exactly what went wrong, and it carries no HTTP details. The Service says "this todo does not exist"; it is the Controller's job to decide that this means a 404.

2. Constructor injection. `TodoService.__init__` **receives** a repository instead of creating one. Notice that the parameter is typed as the abstract `TodoRepository`, not as `InMemoryTodoRepository`. This single detail is what makes the one-line swap possible.

Why the Service never creates its own repository

```
# BAD: the Service is welded to one storage choice.
class TodoService:
    def __init__(self):
        self.repository = InMemoryTodoRepository()

# GOOD: the Service accepts anything that follows the contract.
class TodoService:
    def __init__(self, repository: TodoRepository):
        self.repository = repository
```

3. DTO to model conversion. `create_todo` receives a `TodoCreate` (what the client sent) and builds a full `Todo` from it. The `id`, `completed`, `created_at` and `updated_at` are filled in automatically by the defaults we defined in the model.

services/todo_service.py

```python
from models.todo import Todo, TodoCreate
from repositories.todo_repository import TodoRepository


class TodoNotFoundError(Exception):
    """Raised when a todo does not exist. The Controller turns this
    into a proper HTTP 404 -- the Service knows nothing about HTTP."""
    pass


class TodoService:
    def __init__(self, repository: TodoRepository):
        self.repository = repository

    def create_todo(self, payload: TodoCreate) -> Todo:
        todo = Todo(title=payload.title, description=payload.description)
        return self.repository.add(todo)
```

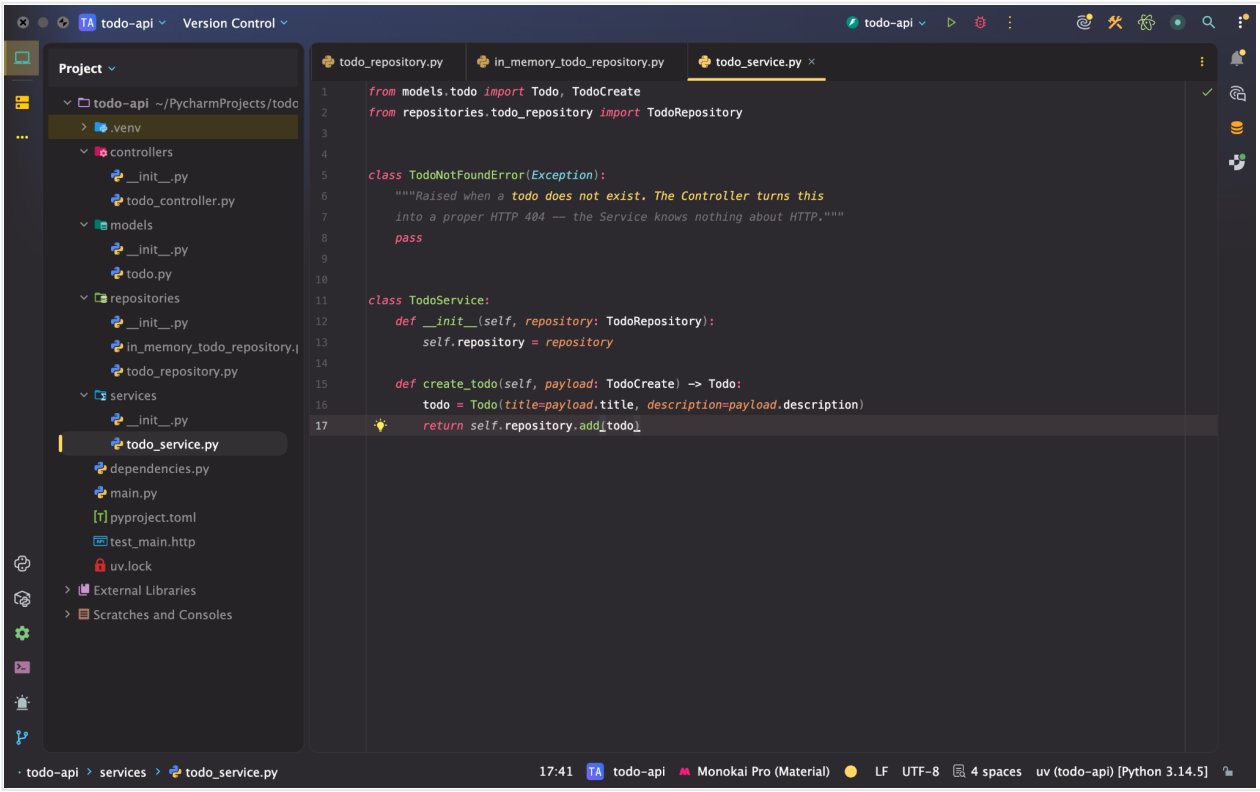


Figure 3.5: services/todo_service.py. The service imports only the contract, never a concrete repository.

Code	What it does
`class TodoNotFoundError(Exception): pass`	A new kind of error. Inheriting from `Exception` makes it raisable and catchable. `pass` means it needs no extra code.
`def __init__(self, repository: TodoRepository)`	Runs when the service is created and stores the repository it was given.
`Todo(title=payload.title, description=payload.description)`	Copies the two client-supplied fields into a full `Todo`. Everything else comes from defaults.

Code	What it does
`return self.repository.add(todo)`	Asks the repository to save it, and returns the saved todo upwards.

3.4 Controller layer (routes / urls)

The Controller is the layer that speaks HTTP. It receives the request, hands it straight to the Service, and returns the result. In FastAPI, a controller is a file of **route functions** attached to a router.

The building blocks

Piece	What it is
`APIRouter(prefix="/todos", tags=["Todos"])`	A mini-application that groups related routes. **prefix** is added in front of every path in this router. **tags** groups the routes under the heading "Todos" in the `/docs` page.
`@router.post("/", ...)`	A **decorator** that registers the function below it as the handler for `POST` requests to `/todos/` (prefix + path).
`response_model=Todo`	The shape of the response. FastAPI validates what you return against it, strips any extra fields and documents it in `/docs`.
`status_code=status.HTTP_201_CREATED`	The status code on success. **201 Created** is the correct answer to a successful create; the default would be 200. Using the named constant is clearer than the number 201.
`payload: TodoCreate`	A parameter whose type is a Pydantic model. FastAPI reads the **JSON body** of the request, validates it as a `TodoCreate`, and passes you the finished object. Bad input never reaches your function.
`service: TodoService = Depends(get_todo_service)`	Asks FastAPI to **inject** a service. `Depends` means "call this function and give me what it returns". We write that function in section 3.5.

How does FastAPI know where each parameter comes from?

If the parameter is...	FastAPI reads it from...	Example in this book
named in the path, like `{todo_id}`	The URL path	`todo_id: UUID` (Step 2)
a Pydantic model	The JSON request body	`payload: TodoCreate`
assigned `Depends(...)`	The result of calling that function	`service: TodoService = Depends(...)`
a simple type not in the path (`int`, `str`)	The query string, like `?limit=10`	Not used yet

controllers/todo_controller.py

```python
from fastapi import APIRouter, Depends, status
from models.todo import Todo, TodoCreate
from services.todo_service import TodoService
from dependencies import get_todo_service

router = APIRouter(prefix="/todos", tags=["Todos"])


@router.post("/", response_model=Todo, status_code=status.HTTP_201_CREATED)
def create_todo(
    payload: TodoCreate,
    service: TodoService = Depends(get_todo_service),
):
    return service.create_todo(payload)
```

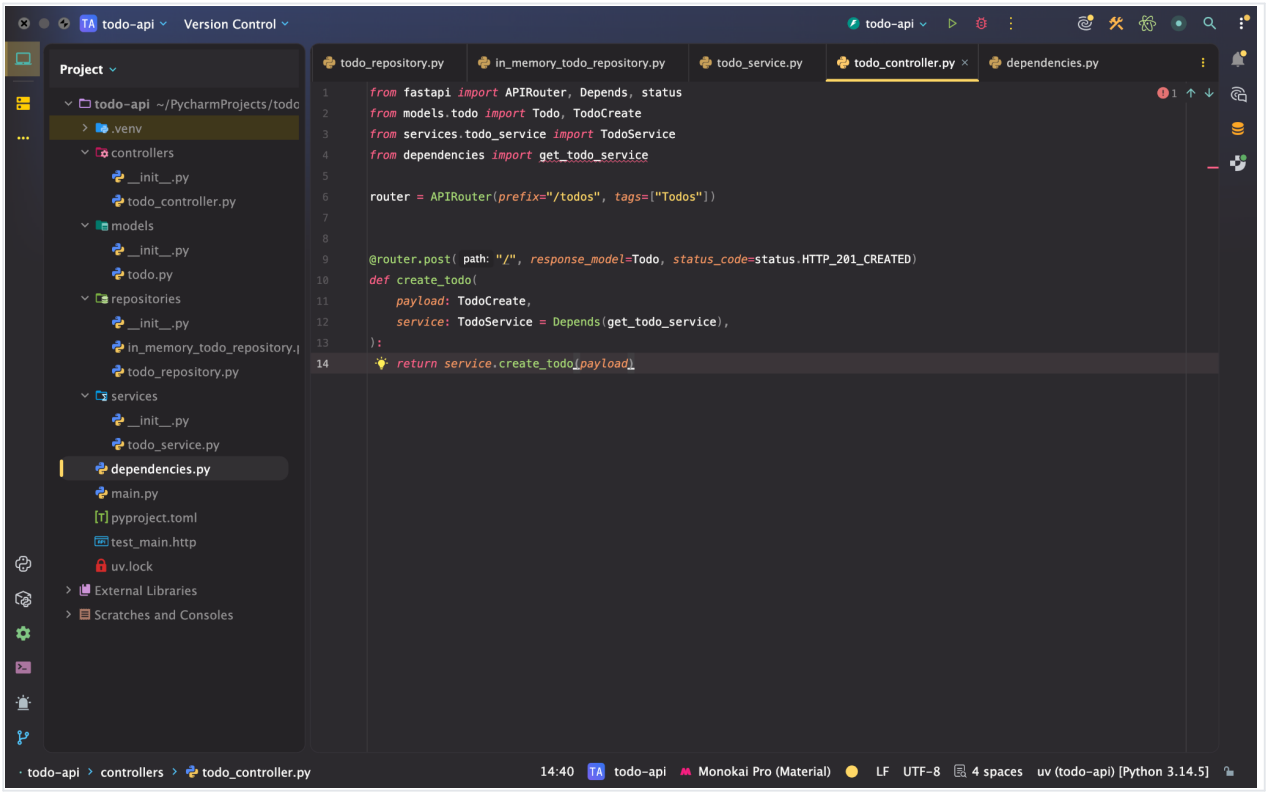


Figure 3.6: controllers/todo_controller.py with the Create route.

> **NOTE**
> The red marker in the screenshot is expected at this point: `dependencies.py` does not exist yet, so the IDE cannot resolve `get_todo_service`. It disappears after the next section.

> **KEY IDEA**
> Look at the body of the route: **one line**. A good controller only translates between HTTP and the Service. If you find yourself writing `if` statements about todos here, that logic belongs in the Service.

3.5 Wire the layers together

`dependencies.py` is the ONE file that knows which concrete repository is in use. This is the file we will change later to swap in MongoDB. Nothing above this point will move.

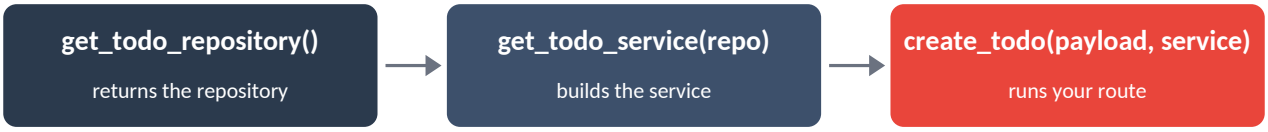
Dependency injection in plain words

Think of a lamp and a wall socket. The lamp does not generate its own electricity; it **receives** power through the plug. It works the same whether the power comes from solar panels or the national grid, because it only relies on the socket's standard shape. Our `TodoService` is the lamp, `TodoRepository` is the socket, and `InMemoryTodoRepository` or `MongoTodoRepository` is the power source.

Dependency injection (DI) means a class receives what it needs from outside instead of building it itself. FastAPI has DI built in through `Depends`.

How FastAPI resolves the chain

When a request reaches `create_todo`, FastAPI sees `Depends(get_todo_service)`. It then looks at the parameters of `get_todo_service`, sees `Depends(get_todo_repository)`, and builds things from the inside out:



The code

dependencies.py

```python
from fastapi import Depends
from repositories.todo_repository import TodoRepository
from repositories.in_memory_todo_repository import InMemoryTodoRepository
from services.todo_service import TodoService

_repository = InMemoryTodoRepository()


def get_todo_repository() -> TodoRepository:
    return _repository


def get_todo_service(
    repository: TodoRepository = Depends(get_todo_repository),
) -> TodoService:
    return TodoService(repository)
```

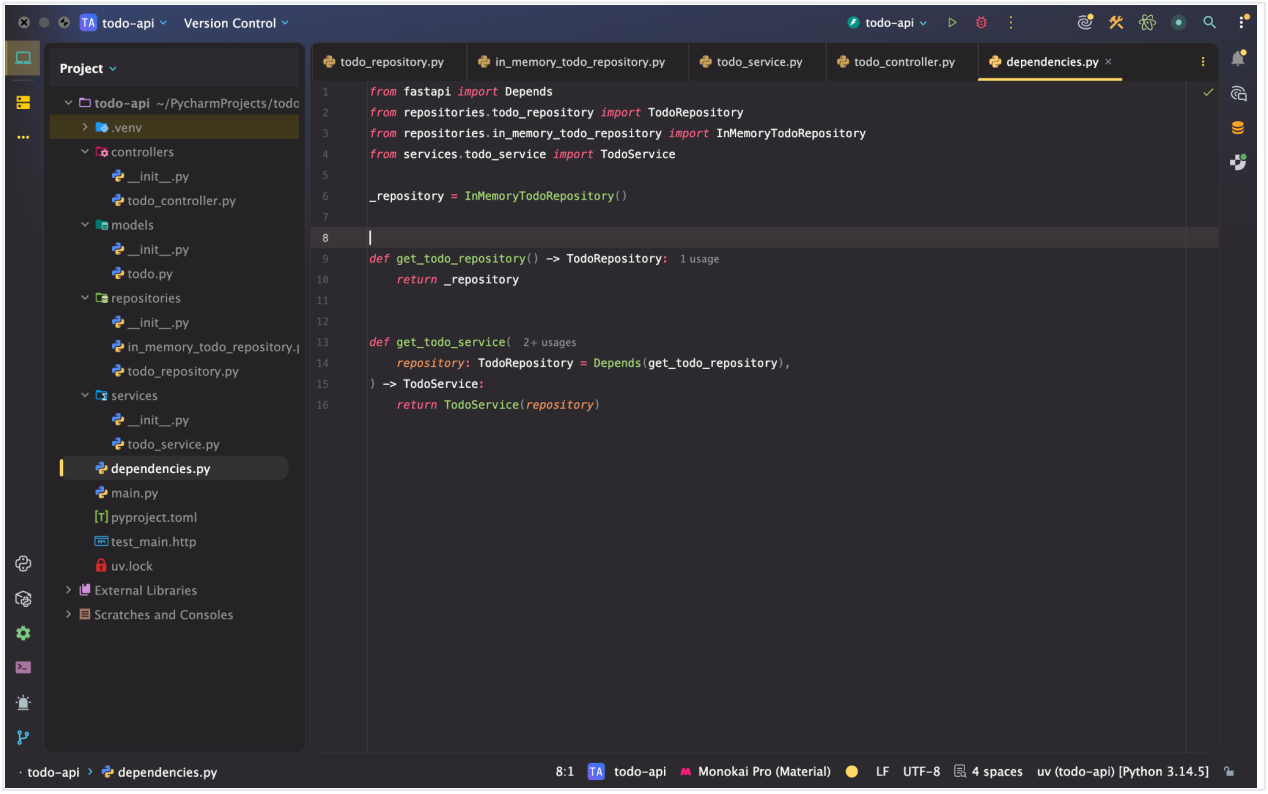


Figure 3.7: dependencies.py, the composition root.

Code	What it does
`_repository = InMemoryTodoRepository()`	Creates **one** repository when the module is first imported. This is the single line that chooses the storage.
`def get_todo_repository() -> TodoRepository`	A tiny function that hands out that repository. Its return type is the **contract**, so callers only assume the contract.
`def get_todo_service(repository = Depends(get_todo_repository))`	FastAPI calls `get_todo_repository` first and passes the result in as `repository`.
`return TodoService(repository)`	Builds the service with that repository. This is DI in action.

> **WATCH OUT**
> The repository is created **once**, at module level, not inside `get_todo_repository()`. If it were created inside the function, every request would get a brand-new empty list and your todos would vanish instantly. The shared `_repository` is what makes the data survive between requests.

> **WHY IT MATTERS**
> Why two functions instead of one? `get_todo_repository` is a **seam**: a place where the system can be cut open and a different part plugged in. In Step 3 we change what it returns, and in Step 4 our tests replace it entirely.

Create the app: main.py

Replace the starter `main.py` that uv created with the following. It creates the FastAPI application and attaches our router to it.

main.py

```
from fastapi import FastAPI
from controllers.todo_controller import router as todo_router

app = FastAPI(title="Todo API")
app.include_router(todo_router)
```

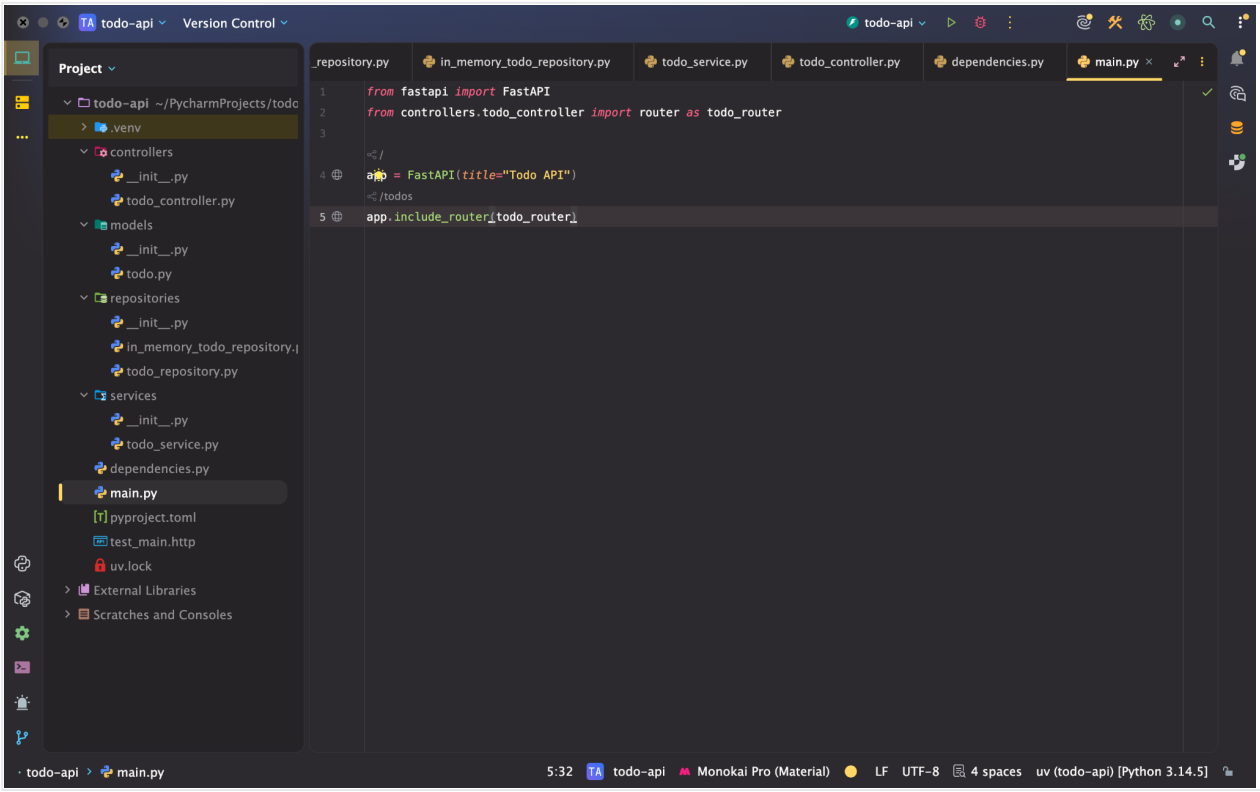


Figure 3.8: main.py. Instantiate FastAPI and include the router.

Code	What it does
`app = FastAPI(title="Todo API")`	Creates the application object. Uvicorn will look for this variable. The title appears at the top of `/docs`.
`from ... import router as todo_router`	Imports the router and renames it with `as`, so that if you later add other routers (users, projects) their names will not clash.
`app.include_router(todo_router)`	Attaches all the routes of the router (with the `/todos` prefix) to the application.

`main.py` is deliberately tiny. It is the **entry point**: it assembles the pieces and contains no logic of its own.

3.6 Run it and test the Create endpoint

Terminal

```
uv run uvicorn main:app --reload
```

Part	Meaning
`uv run`	Runs the command inside the project's virtual environment.
`uvicorn`	The server program.
`main:app`	Find the file `main.py` and, inside it, the variable `app`.
`--reload`	Restart the server automatically whenever you save a file. Use it while developing only.



Figure 3.9: Uvicorn starts. "Application startup complete" means the server is ready.

The server is now listening on `http://127.0.0.1:8000` (your own computer, port 8000). Leave this terminal running and use a second one for other commands.

Try it in the interactive docs

FastAPI generates a testing page for free. Open `http://127.0.0.1:8000/docs` in your browser. This is **Swagger UI**, built from your routes and your models. Expand `POST /todos/`, click **Try it out**, and send:

Request body

```
{
  "title": "Buy milk",
  "description": "2 liters, whole"
}
```

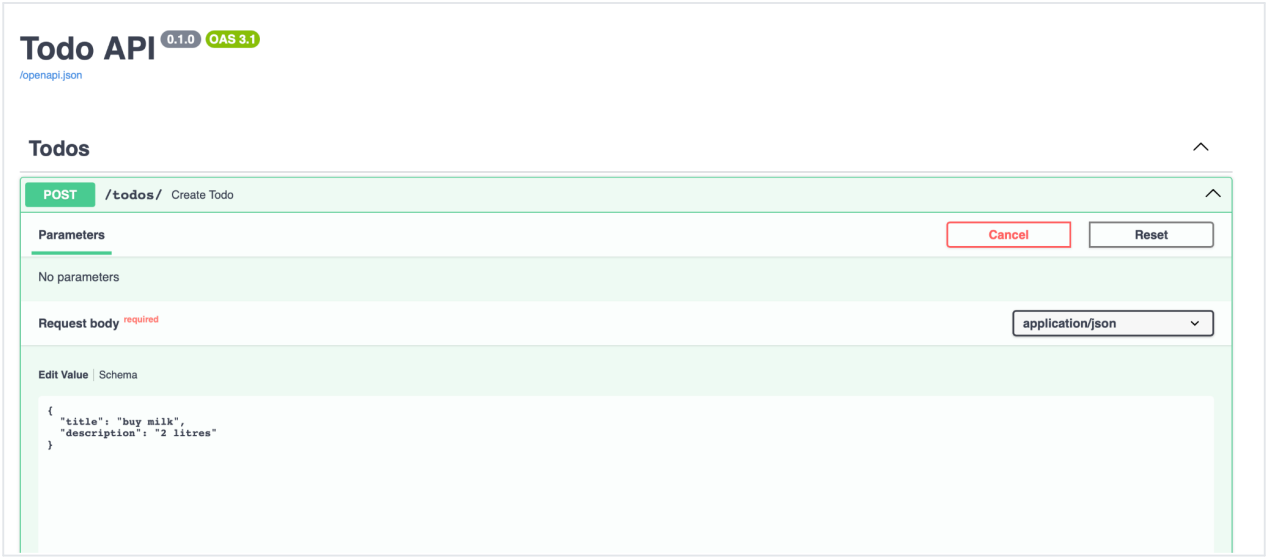


Figure 3.10: Swagger UI with POST /todos/ expanded and the request body filled in.



Figure 3.11: The server log. The last line shows POST /todos/ answered with 201 Created.

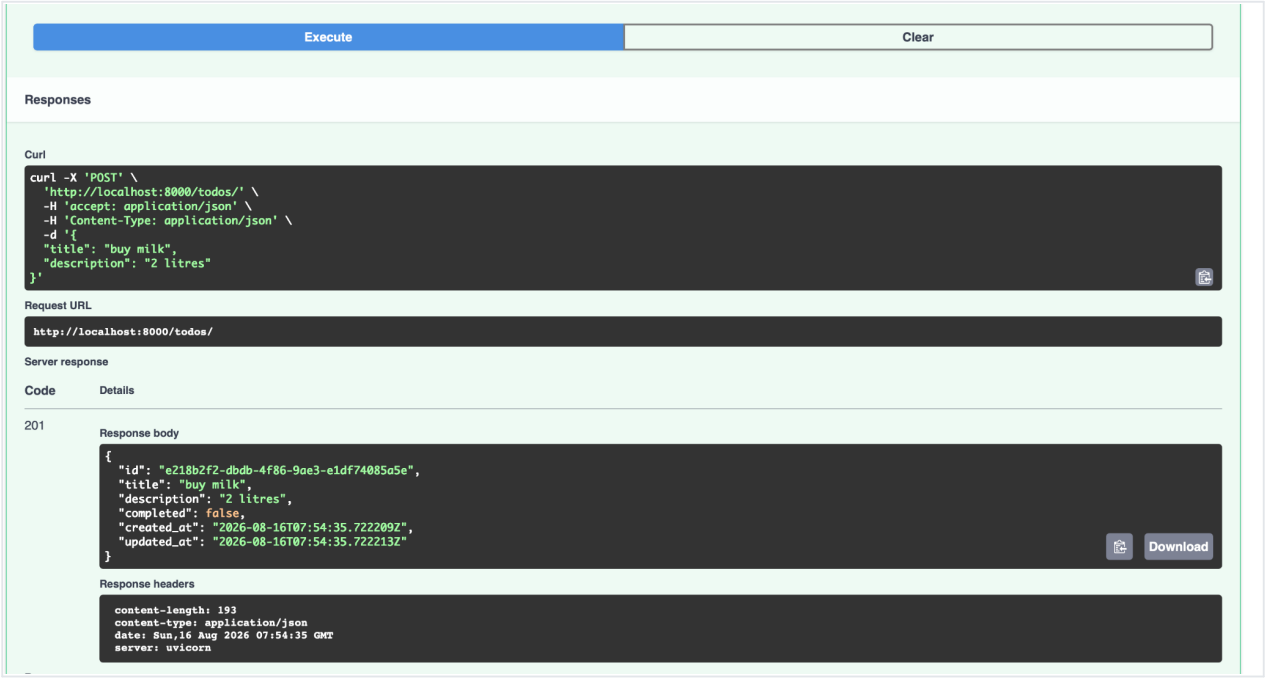


Figure 3.12: The response: the equivalent curl command, status code 201, the new todo and the response headers.

Or from a second terminal, using `curl`:

Terminal

```
curl -X POST http://127.0.0.1:8000/todos/ \
  -H "Content-Type: application/json" \
  -d '{"title": "Buy milk", "description": "2 liters, whole"}'
```

curl part	Meaning
`-X POST`	Use the HTTP method POST.
`-H "Content-Type: application/json"`	A header telling the server the body is JSON.
`-d '{...}'`	The request body (the data to send).

NOTE

What you should see: a 201 response with a full Todo object: a generated `id`, your `title` and `description`, `completed: false`, and timestamps. Restart the server and the todo disappears. That is expected, since the list lives only in memory. Step 3 fixes that.

Which code ran? Trace the request

Stage	Code that ran
Routing	FastAPI matched `POST /todos/` to `create_todo` in `controllers/todo_controller.py`.
Validation	The JSON became a `TodoCreate`. Had the title been missing, FastAPI would have answered 422 and **never called** your function.
Injection	`get_todo_repository()` returned `_repository`; `get_todo_service(...)` wrapped it in a `TodoService`.
Service	`create_todo` built a `Todo` with a new `id` and timestamps and called `repository.add(todo)`.
Repository	`InMemoryTodoRepository.add` appended it to `self._todos`.
Response	The returned `Todo` was checked against `response_model=Todo`, converted to JSON and sent with status 201.

TRY IT

Send bad input and watch what happens: (1) an empty body `{}`, (2) `{"title": ""}`, (3) a title of 201 characters. Each returns **422 Unprocessable Entity** with a message saying which field failed and why. You wrote no validation code for this. The model did it.

A 422 response (abridged)

```
{"detail": [{"type": "missing", "loc": ["body", "title"], "msg": "Field required", "input": {}}]}
```

Checkpoint

You now have a vertical slice through all four layers. Before continuing, make sure you can answer, in your own words:

1. Why is `TodoCreate` a separate class from `Todo`?
2. Which file would you edit to change where todos are stored? Which files would you **not** need to edit?
3. Why does `TodoService` ask for a `TodoRepository` and not for an `InMemoryTodoRepository`?
4. Where does the 422 error come from when the title is empty?

4. Step 2 — Complete the Full CRUD

CRUD stands for **C**reate, **R**ead, **U**pdate, **D**elete: the four things almost every application does with its data. The repository already declares every method we need. We now add the remaining Service methods and Controller routes: list, get one, update, delete.

Notice what we do **not** touch: the Model and the Repository contract. That is the layered design paying off. New features mostly add code in the upper layers.

4.1 Add these methods to the Service

Replace the contents of `services/todo_service.py` with the full file below. The new parts are the extra imports and four new methods.

services/todo_service.py (full file)

```python
from typing import List
from uuid import UUID
from models.todo import Todo, TodoCreate, TodoUpdate
from repositories.todo_repository import TodoRepository


class TodoNotFoundError(Exception):
    """Raised when a todo does not exist. The Controller turns this
    into a proper HTTP 404 -- the Service knows nothing about HTTP."""
    pass


class TodoService:
    def __init__(self, repository: TodoRepository):
        self.repository = repository

    def create_todo(self, payload: TodoCreate) -> Todo:
        todo = Todo(title=payload.title, description=payload.description)
        return self.repository.add(todo)

    def list_todos(self) -> List[Todo]:
        return self.repository.list()

    def get_todo(self, todo_id: UUID) -> Todo:
        todo = self.repository.get(todo_id)
        if todo is None:
            raise TodoNotFoundError(f"Todo {todo_id} not found")
        return todo

    def update_todo(self, todo_id: UUID, payload: TodoUpdate) -> Todo:
        data = payload.model_dump(exclude_unset=True)
        todo = self.repository.update(todo_id, data)
        if todo is None:
            raise TodoNotFoundError(f"Todo {todo_id} not found")
        return todo

    def delete_todo(self, todo_id: UUID) -> None:
        deleted = self.repository.delete(todo_id)
        if not deleted:
            raise TodoNotFoundError(f"Todo {todo_id} not found")
```

> **WATCH OUT**
> The screenshot below is scrolled down, so the imports at the top of the file are hidden. Use the listing above for the complete file.

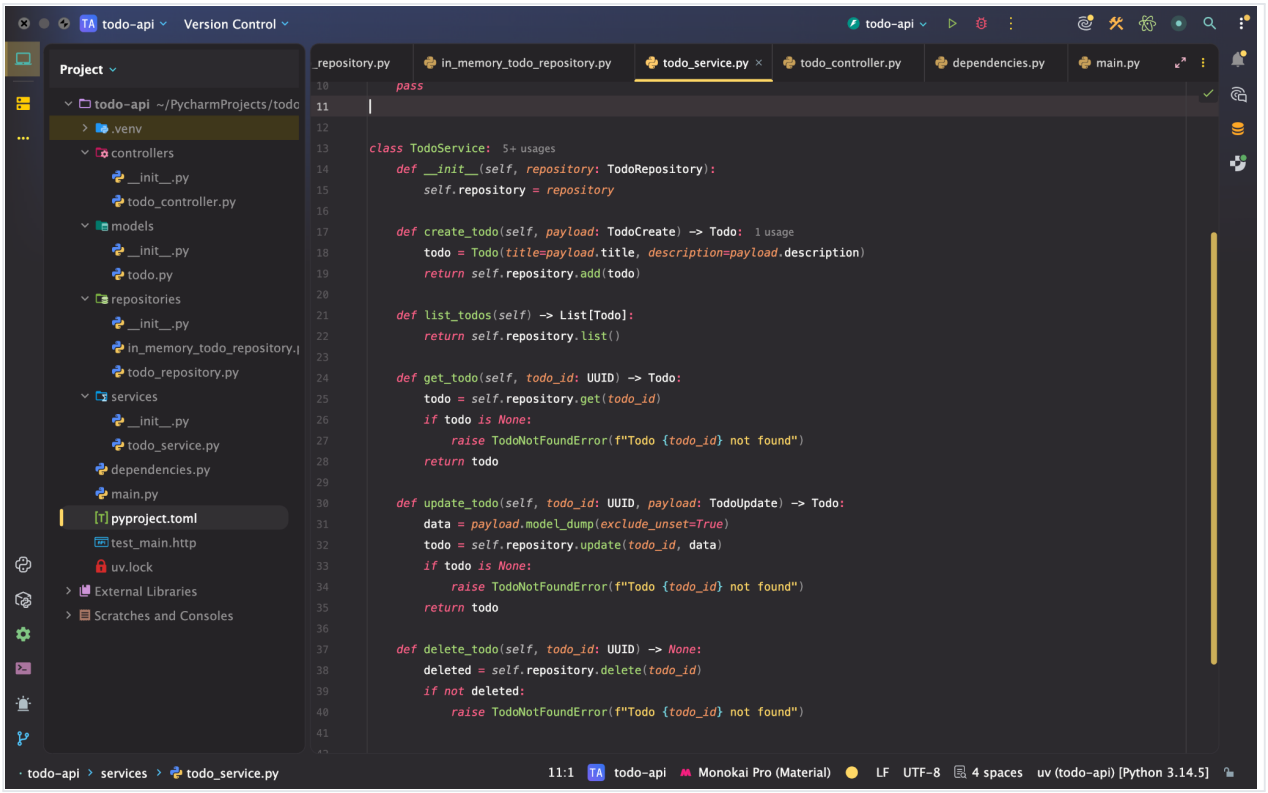


Figure 4.1: services/todo_service.py with all five methods.

The pattern: ask, check, raise

`get_todo`, `update_todo` and `delete_todo` all follow the same three beats:

1. **Ask** the repository to do the work.
2. **Check** the answer: did it find something? (`None` or `False` means no.)
3. **Raise** a `TodoNotFoundError` if not, otherwise return the result.

This is the business rule "you cannot read, change or delete a todo that does not exist", written exactly once, in the layer where rules live.

A closer look at update_todo and exclude_unset

`update_todo` is the most interesting method. The client may send only the fields they want to change (this is called a **partial update**, and the HTTP method for it is `PATCH`).
`payload.model_dump(exclude_unset=True)` turns the `TodoUpdate` into a dictionary containing **only the fields the client actually sent**.

Client sends	TodoUpdate becomes	`model_dump(exclude_unset=True)`	Effect
`{"completed": true}`	`completed=True`, others `None`	`{"completed": True}`	Only `completed` changes

Client sends	TodoUpdate becomes	`model_dump(exclude_unset=True)`	Effect
`{"title": "New", "completed": true}`	two fields set	`{"title": "New", "completed": True}`	Both change
`{"description": null}`	`description=None` (set explicitly)	`{"description": None}`	The description is **cleared**
`{}`	nothing set	`{}`	Nothing changes

> **WATCH OUT**
> Without `exclude_unset=True`, `model_dump()` would include every field, and the ones the client left out would arrive as `None`. The repository would then overwrite the existing title and description with `None`. A tick-box request would wipe the todo. Using `exclude_none=True` instead would also be wrong: the client could then never clear a field on purpose.

4.2 Add these routes to the Controller

Replace `controllers/todo_controller.py` with the full file below. Each route follows the same shape as the Create route: a decorator, typed parameters, one call to the Service.

controllers/todo_controller.py (full file)

```
from fastapi import APIRouter, Depends, HTTPException, status
from uuid import UUID
from models.todo import Todo, TodoCreate, TodoUpdate
from services.todo_service import TodoService, TodoNotFoundError
from dependencies import get_todo_service

router = APIRouter(prefix="/todos", tags=["Todos"])


@router.post("/", response_model=Todo, status_code=status.HTTP_201_CREATED)
def create_todo(
    payload: TodoCreate,
    service: TodoService = Depends(get_todo_service),
):
    return service.create_todo(payload)


@router.get("/", response_model=list[Todo])
def list_todos(service: TodoService = Depends(get_todo_service)):
    return service.list_todos()


@router.get("/{todo_id}", response_model=Todo)
def get_todo(
    todo_id: UUID,
    service: TodoService = Depends(get_todo_service),
):
    try:
        return service.get_todo(todo_id)
    except TodoNotFoundError as e:
        raise HTTPException(status_code=404, detail=str(e))
```

```python
@router.patch("/{todo_id}", response_model=Todo)
def update_todo(
    todo_id: UUID,
    payload: TodoUpdate,
    service: TodoService = Depends(get_todo_service),
):
    try:
        return service.update_todo(todo_id, payload)
    except TodoNotFoundError as e:
        raise HTTPException(status_code=404, detail=str(e))


@router.delete("/{todo_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_todo(
    todo_id: UUID,
    service: TodoService = Depends(get_todo_service),
):
    try:
        service.delete_todo(todo_id)
    except TodoNotFoundError as e:
        raise HTTPException(status_code=404, detail=str(e))
```

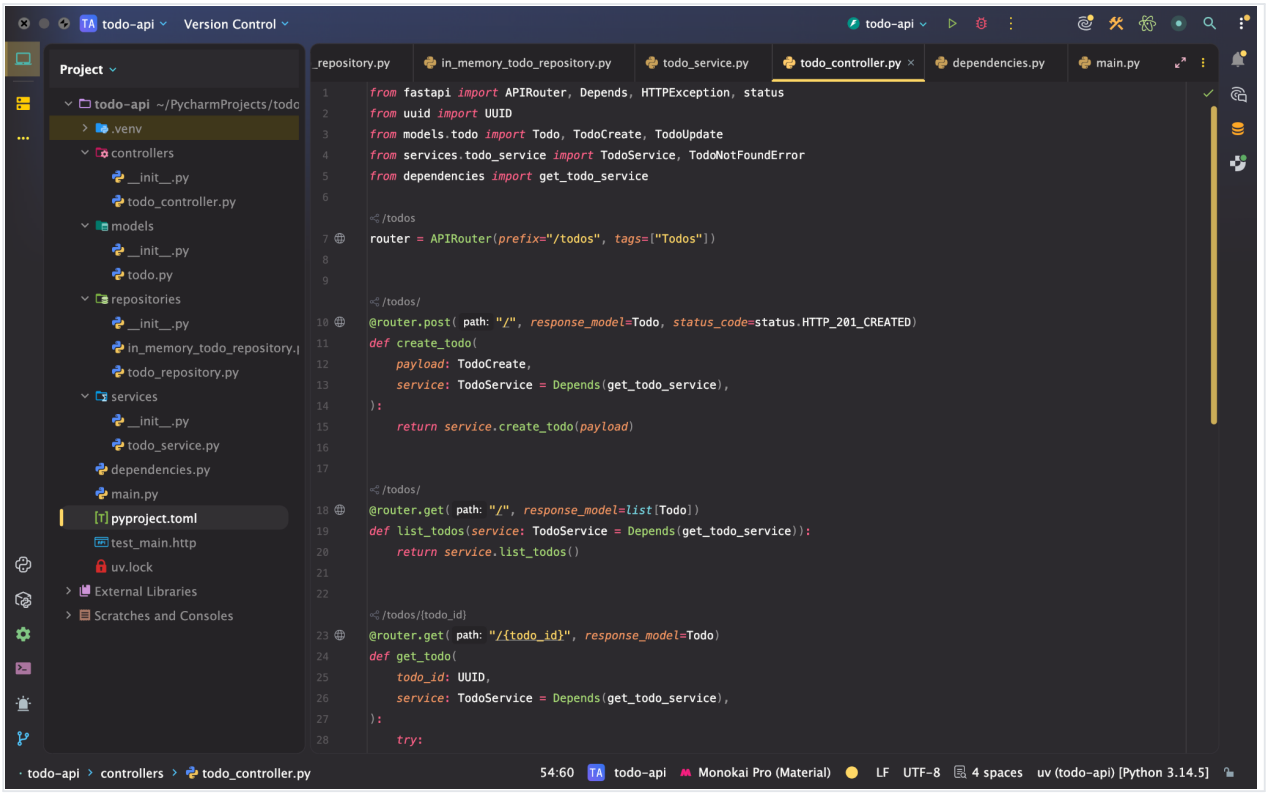


Figure 4.2: controllers/todo_controller.py (top of the file).

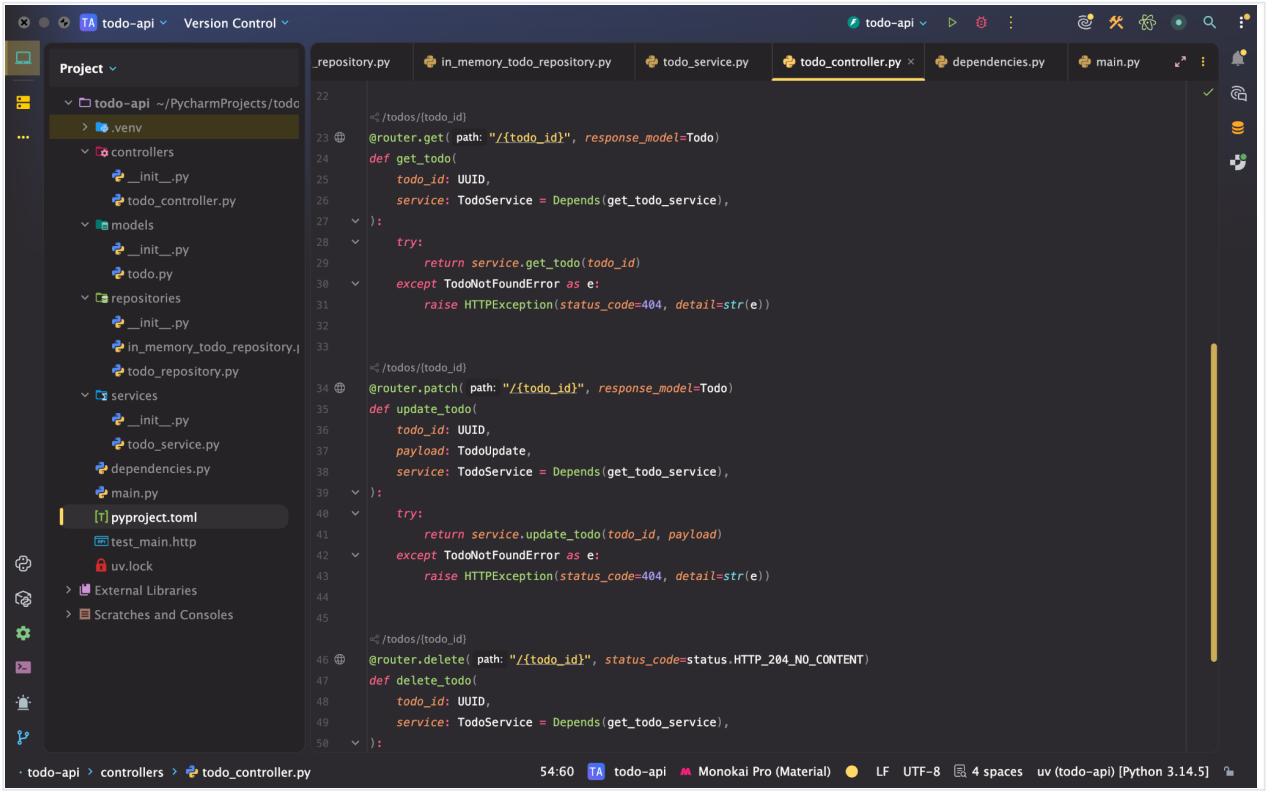


Figure 4.3: controllers/todo_controller.py (get, update and delete routes).



Figure 4.4: the end of the file: the delete route.

The five endpoints at a glance

Method	URL	Purpose	Success	Errors
POST	/todos/	Create a todo	201 + the todo	422 bad body
GET	/todos/	List all todos	200 + a list	(none)
GET	/todos/{todo_id}	Fetch one todo	200 + the todo	404 missing, 422 bad id
PATCH	/todos/{todo_id}	Change some fields	200 + the updated todo	404, 422
DELETE	/todos/{todo_id}	Remove a todo	204, no body	404, 422

New ideas in this file

Idea	Explanation
Path parameters `/{todo_id}`	The part in braces is a variable. Whatever the client puts there is passed to the function argument of the same name.
`todo_id: UUID`	FastAPI converts the text in the URL into a `UUID` and **validates it**. A request to `/todos/hello` is rejected with 422 before your code runs.
`response_model=list[Todo]`	The response is a **list** of `Todo` objects. Here we can use the built-in `list[...]`, because this is a normal function, not inside the repository class.
`PATCH` vs `PUT`	`PATCH` changes **some** fields; `PUT` replaces the **whole** resource. Our `TodoUpdate` (all fields optional) fits PATCH.
`status_code=204` (No Content)	Success with an intentionally empty body. A delete has nothing useful to return, so the route returns nothing.
`try ... except TodoNotFoundError as e`	Catch the Service's plain Python error and translate it.
`raise HTTPException(status_code=404, detail=str(e))`	`HTTPException` is FastAPI's way to stop and send an error response. `str(e)` is the message we wrote in the Service. The client receives `{"detail": "Todo ... not found"}`.

> **KEY IDEA**
>
> **Notice:** the Service raises a plain Python `TodoNotFoundError`. It has never heard of HTTP. Only the Controller knows that "not found" means a 404. That is the layering rule in action. If tomorrow you add a command-line tool or a gRPC interface, the same Service works unchanged, because the HTTP translation lives only in the Controller.

Common HTTP status codes in this project

Code	Name	When we use it
200	OK	A successful GET or PATCH.
201	Created	A successful POST that created something.
204	No Content	A successful DELETE (nothing to send back).
404	Not Found	The Service raised `TodoNotFoundError`.
422	Unprocessable Entity	FastAPI rejected the input (missing field, wrong type, bad UUID).

> **NOTE**
>
> The `try/except` block repeats in three routes. That is acceptable while you are learning. FastAPI can also register **one** exception handler for `TodoNotFoundError` that returns the 404 for all routes. Making that change is one of the exercises at the end of the book.

4.3 Exercise all five endpoints

Restart uvicorn (or let `--reload` pick it up) and exercise all five endpoints from `/docs`:

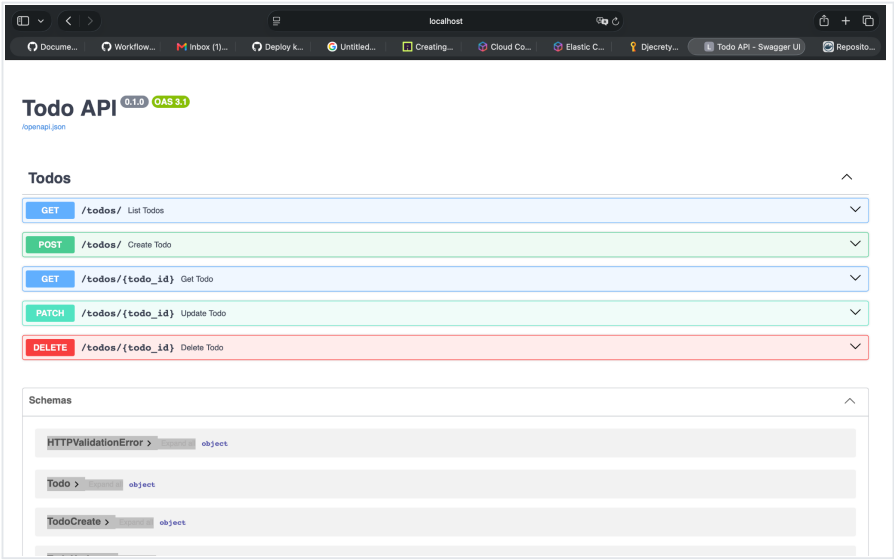


Figure 4.5: Swagger UI now lists all five endpoints. Under Schemas you can see Todo, TodoCreate and TodoUpdate: our DTOs, documented automatically.

Now create a couple of todos, list them, fetch one by id, patch it, then delete it. Use this checklist:

#	Request	What you should see
1	`POST /todos/` with `{"title": "Buy milk"}`	201 and a todo with an `id`. **Copy that id.**
2	`POST /todos/` with `{"title": "Walk the dog"}`	201 and a second todo.
3	`GET /todos/`	200 and a list with both todos.
4	`GET /todos/{id}` with the id from step 1	200 and the "Buy milk" todo.
5	`PATCH /todos/{id}` with `{"completed": true}`	200. `completed` is now `true`; title and description are untouched.
6	`DELETE /todos/{id}`	204 with an empty body.
7	`GET /todos/{id}` again	404 with `{"detail": "Todo ... not found"}`.
8	`GET /todos/hello`	422: "hello" is not a valid UUID.

> **NOTE**
> **A small gap to spot.** After step 5, look at `updated_at`: the in-memory `update` does not refresh it, but the MongoDB version in Step 3 does. Two implementations of one contract should behave the same way. Fixing this is exercise 1 at the end of the book.

5. Step 3 — Swap the List for MongoDB (pymongo)

This is the payoff. We add a new repository that talks to MongoDB instead of a list, then change ONE line in `dependencies.py`. The Model, Service and Controller do not change at all.

5.1 Install pymongo and start MongoDB

Why a database?

A Python list lives in the memory of the running program. When the server restarts, the list is gone. A **database** stores data on disk, outside your program, so it survives restarts and can be shared.

What is MongoDB?

MongoDB is a **document database**. Instead of tables with rows and columns, it stores **documents**: JSON-like objects. The terms map like this:

In SQL databases	In MongoDB	In our project
Database	Database	`todo_app`
Table	Collection	`todos`
Row	Document	One todo
Primary key	`_id` field	The todo's UUID, as text

pymongo is MongoDB's official Python driver: the library our code uses to talk to the database.

If you have Docker, the fastest way to get a local MongoDB running:

Terminal

```
uv add pymongo

docker run -d -p 27017:27017 --name todo-mongo mongo:7
```

Part	Meaning
`docker run -d`	Start a container in the background (**d**etached).
`-p 27017:27017`	Make port 27017 inside the container available as port 27017 on your computer. 27017 is MongoDB's default port.
`--name todo-mongo`	Give the container a name, so you can manage it later.
`mongo:7`	The image to run: MongoDB version 7.

Later you can stop it with `docker stop todo-mongo` and start it again with `docker start todo-mongo`. Check that it is running with `docker ps`.

No Docker? Install MongoDB Community Server directly and make sure it is running on the default port, 27017.

5.2 Database connection

We put the connection in its own file, so that every other file can simply import a ready-made collection.

database.py

```
import os
from pymongo import MongoClient

MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017")

client = MongoClient(MONGO_URI)
database = client["todo_app"]
todos_collection = database["todos"]
```

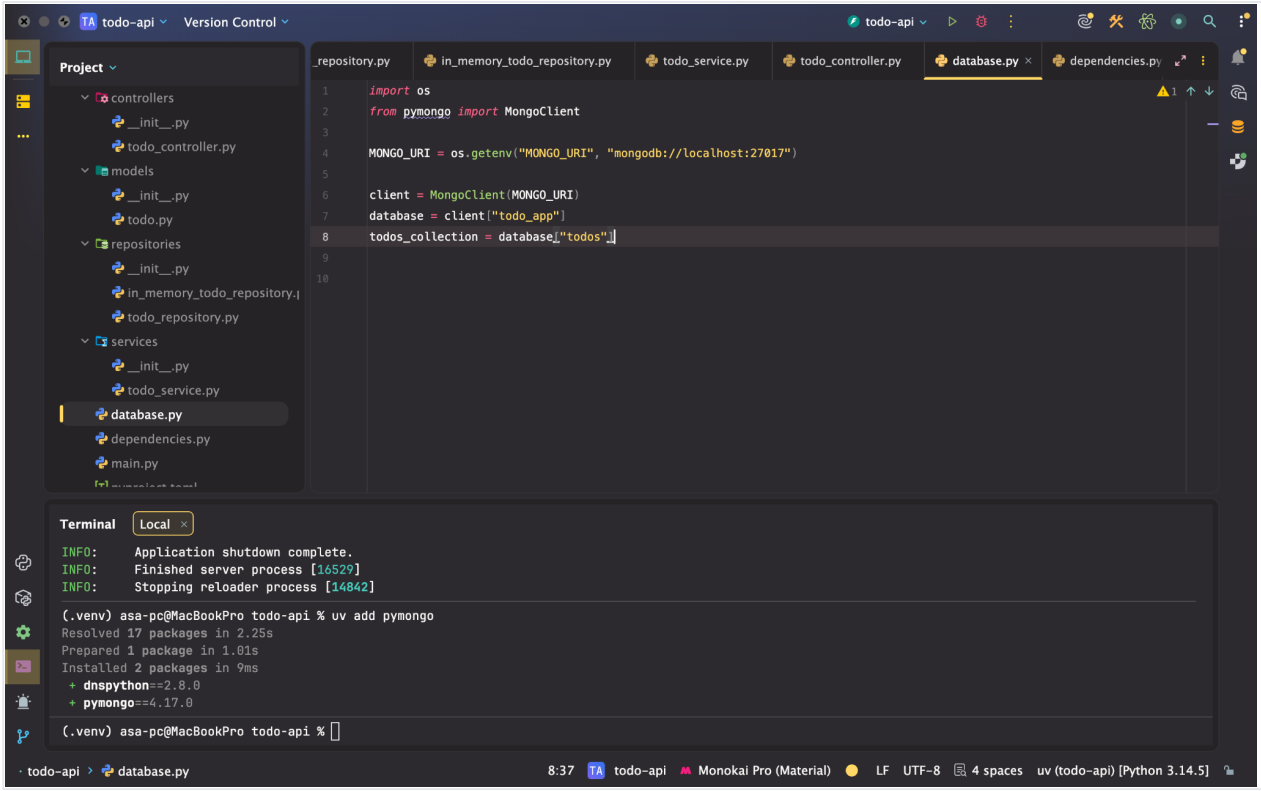


Figure 5.1: database.py. The terminal below shows uv add pymongo completing.

Code	What it does
`os.getenv("MONGO_URI", "mongodb://localhost:27017")`	Reads an **environment variable** called `MONGO_URI`. If it is not set, it uses the default after the comma: a MongoDB on your own computer. In production you would set the variable to point at a real server, **without editing any code** (and without putting passwords in your source files).
`MongoClient(MONGO_URI)`	Creates the client object that manages connections. Create **one** for the whole application and share it. It connects lazily: no connection is made until the first real operation.
`client["todo_app"]`	Selects the database named `todo_app`. MongoDB creates it the first time you store something in it.
`database["todos"]`	Selects the collection `todos`. Also created on first use.

5.3 MongoDB repository

This class implements the exact same `TodoRepository` contract from Step 1, in its own file, just like `InMemoryTodoRepository`. Compare its method signatures side by side with `in_memory_todo_repository.py`. They are identical. Only what happens **inside** each method is different.

Entities versus documents

Our code works with `Todo` objects (the **entity**). MongoDB works with dictionaries (the **document**). The repository is the translator between the two, which is why it has two small helper methods:

Helper	Direction	What it does
`_to_document(todo)`	`Todo` → dict	Calls `todo.model_dump()` to get a dictionary, then renames the key `id` to `_id` and stores it as text with `str(...)`.
`_to_entity(document)`	dict → `Todo`	Copies the dictionary, renames `_id` back to `id`, and builds a `Todo` with `Todo(**document)`. Pydantic converts the text back into a `UUID`.

Why `_id`? `_id` is MongoDB's required primary-key field. If you do not supply one, MongoDB invents an `ObjectId` that our `Todo` model has never heard of. By supplying our own UUID as `_id`, the id a client sees is the same id the database uses.

Why as text? pymongo does not encode a native Python `UUID` unless you configure how it should be stored. Converting to a string with `str(...)` is the simplest option, and Pydantic converts it back on the way out.

The code

repositories/mongo_todo_repository.py

```python
from datetime import datetime, timezone
from typing import List, Optional
from uuid import UUID
from pymongo import ReturnDocument
from pymongo.collection import Collection
from models.todo import Todo
from repositories.todo_repository import TodoRepository


class MongoTodoRepository(TodoRepository):
    """Same contract as InMemoryTodoRepository, backed by MongoDB."""

    def __init__(self, collection: Collection):
        self._collection = collection

    def add(self, todo: Todo) -> Todo:
        self._collection.insert_one(self._to_document(todo))
        return todo

    def get(self, todo_id: UUID) -> Optional[Todo]:
        document = self._collection.find_one({"_id": str(todo_id)})
        return self._to_entity(document) if document else None

    def list(self) -> List[Todo]:
        return [self._to_entity(doc) for doc in self._collection.find()]
```

```python
    def update(self, todo_id: UUID, data: dict) -> Optional[Todo]:
        data["updated_at"] = datetime.now(timezone.utc)
        document = self._collection.find_one_and_update(
            {"_id": str(todo_id)},
            {"$set": data},
            return_document=ReturnDocument.AFTER,
        )
        return self._to_entity(document) if document else None

    def delete(self, todo_id: UUID) -> bool:
        result = self._collection.delete_one({"_id": str(todo_id)})
        return result.deleted_count == 1

    @staticmethod
    def _to_document(todo: Todo) -> dict:
        document = todo.model_dump()
        document["_id"] = str(document.pop("id"))
        return document

    @staticmethod
    def _to_entity(document: dict) -> Todo:
        document = dict(document)
        document["id"] = document.pop("_id")
        return Todo(**document)
```

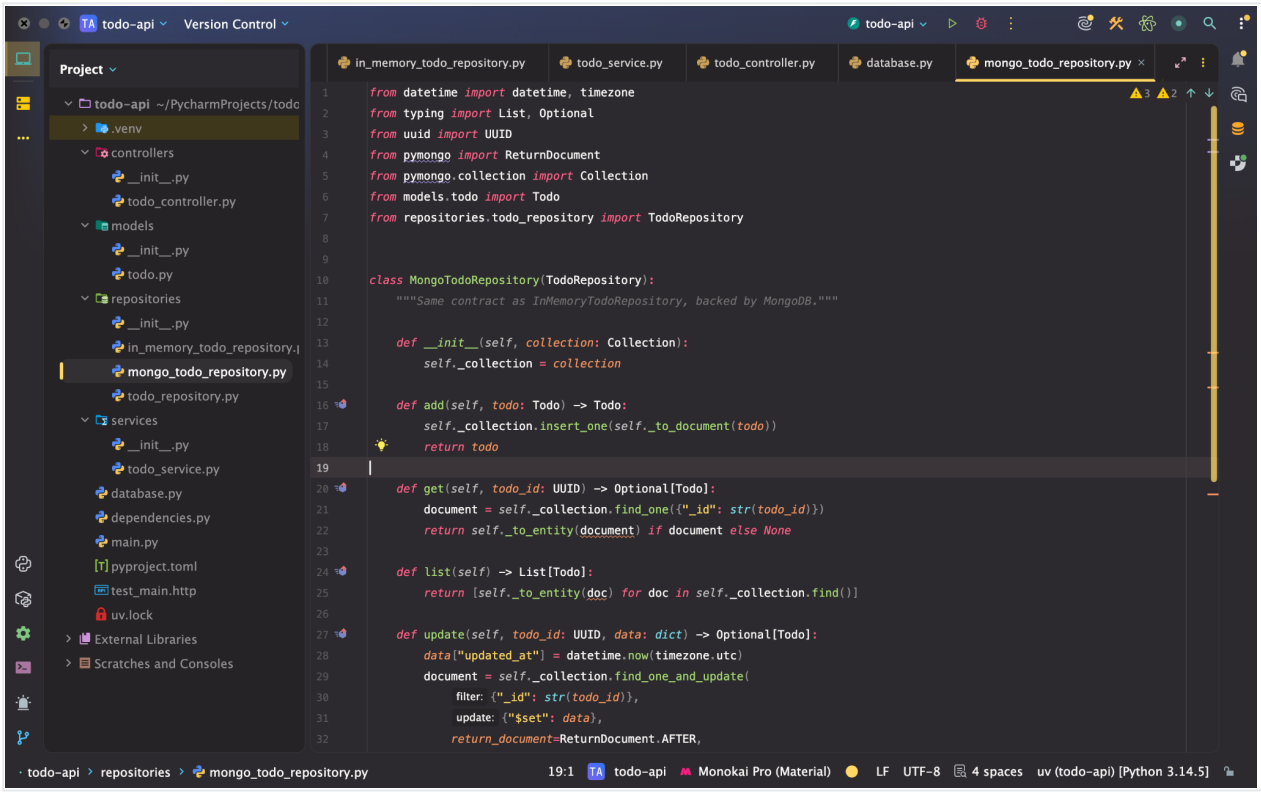


Figure 5.2: mongo_todo_repository.py (top of the file).

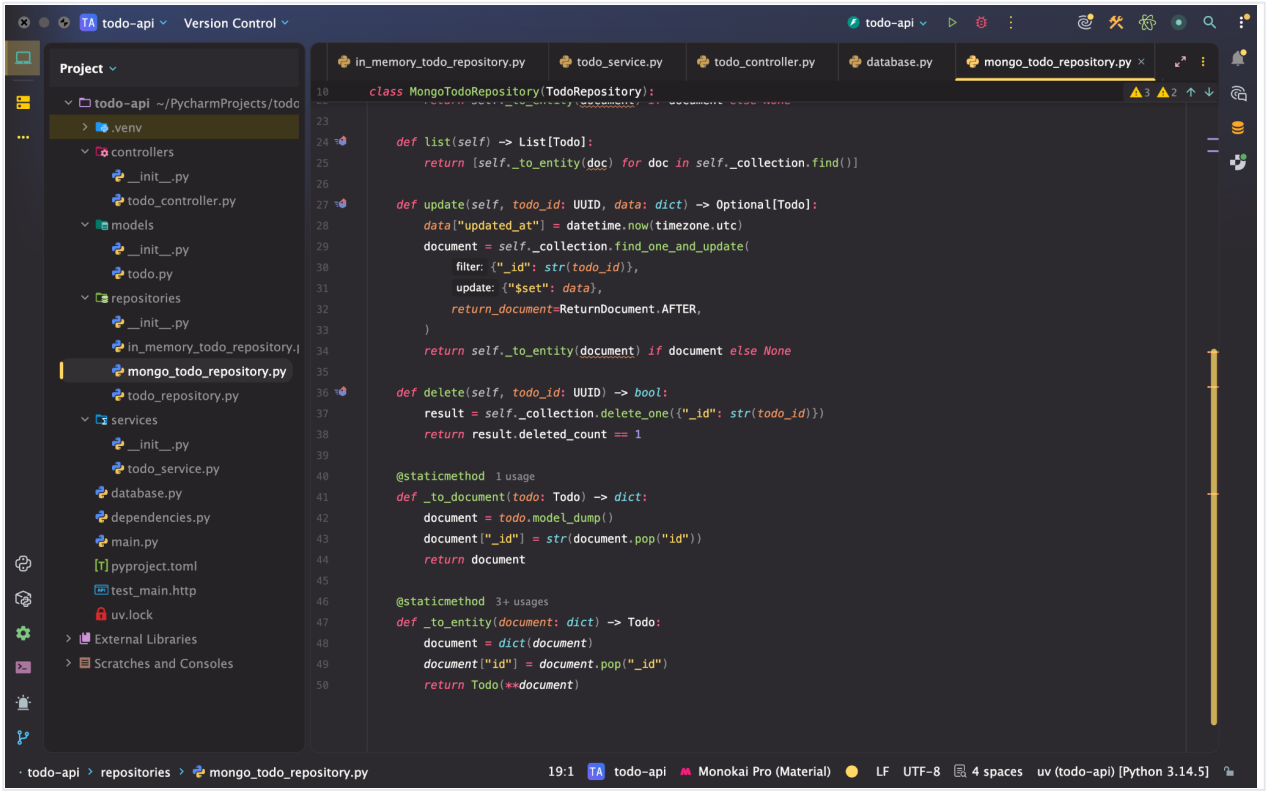


Figure 5.3: mongo_todo_repository.py (update, delete and the two mapping helpers).

Method by method

Method	In-memory version	MongoDB version
`add`	`self._todos.append(todo)`	`insert_one(...)` saves the document built by `_to_document`.
`get`	Loop over the list	`find_one({"_id": ...})` returns one document or `None`. We convert it to a `Todo`, or return `None`.
`list`	Return the list	`find()` returns a cursor over all documents; a list comprehension converts each one.
`update`	`model_copy(update=data)`	`find_one_and_update(filter, {"$set": data}, ...)` finds and changes the document in one step.
`delete`	`remove(todo)`	`delete_one(...)`; `deleted_count == 1` tells us whether something was really deleted.

The update call in detail

Piece	Meaning
`{"_id": str(todo_id)}`	The **filter**: which document to change.
`{"$set": data}`	The **update**. `$set` changes only the listed fields and leaves all the others alone. Without `$set`, MongoDB would replace the entire document.
`return_document=ReturnDocument.AFTER`	By default MongoDB returns the document as it was **before** the change. `AFTER` returns the new version, which is what we want to give back.
`data["updated_at"] = datetime.now(timezone.utc)`	Refreshes the modification time on every update.

Why __init__ takes the collection

`MongoTodoRepository` receives its `collection` through the constructor. It does not import `database.py` itself. That keeps it flexible: you could pass a different collection (for example, one in a test database) without changing the class. Same principle as before: receive your dependencies, do not create them.

> NOTE
> **A detail about dates.** pymongo returns datetimes **without** a time-zone by default (they are still UTC). If you ever need time-zone-aware values back from MongoDB, create the client with `MongoClient(MONGO_URI, tz_aware=True)`. It makes no difference for this project.

5.4 The one-line swap

Open `dependencies.py` and replace the repository line. This is the only edit needed anywhere in the project:

dependencies.py (updated)

```python
from fastapi import Depends
from database import todos_collection
from repositories.todo_repository import TodoRepository
from repositories.mongo_todo_repository import MongoTodoRepository
from services.todo_service import TodoService

_repository = MongoTodoRepository(todos_collection)
# was: _repository = InMemoryTodoRepository()


def get_todo_repository() -> TodoRepository:
    return _repository


def get_todo_service(
    repository: TodoRepository = Depends(get_todo_repository),
) -> TodoService:
    return TodoService(repository)
```

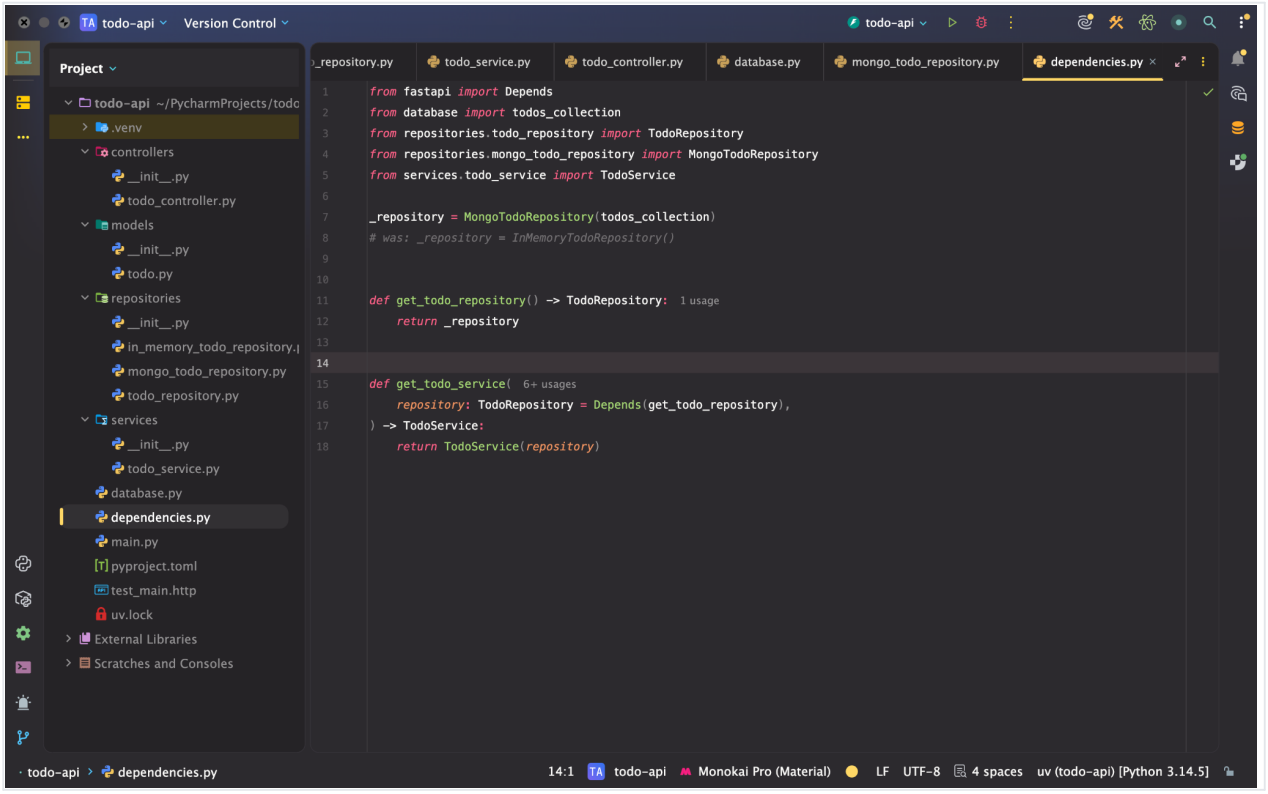
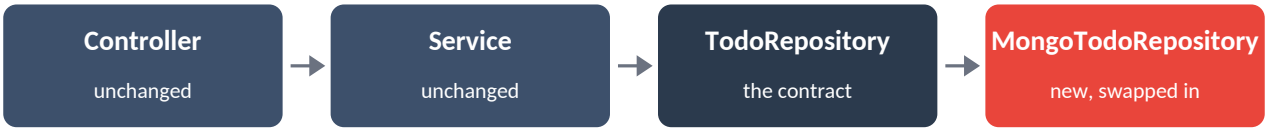


Figure 5.4: dependencies.py after the swap. One line chose a different implementation.

Strictly speaking, the change is **one line of logic** (the assignment to `_repository`) plus the `import` lines that bring in the new names. Every function below it is untouched.

Why nothing else had to change

File	Why it did not need an edit
`models/todo.py`	Describes data. Knows nothing about storage.
`services/todo_service.py`	Depends on the `TodoRepository` **contract**. A `MongoTodoRepository` honours that contract, so the Service cannot tell the difference.
`controllers/todo_controller.py`	Never sees a repository at all. It only knows the Service.
`main.py`	Only assembles the app.



> **KEY IDEA**
> This is what "depend on abstractions, not on concrete classes" buys you. The decision about **which** storage to use is made in exactly one place.

What you should see: restart uvicorn and repeat the exact same requests from Step 2 in `/docs`. They behave identically, except now, if you restart the server, your todos are still there. You can also open the database with MongoDB Compass or `mongosh` and see the documents in the `todo_app.todos` collection.

mongosh

```
use todo_app
db.todos.find()
```

A stored document looks like this (values will differ):

One document in the todos collection

```
{
  _id: '3f2b1c9e-5d4a-4f1e-9a57-2c8d0e6b7a11',
  title: 'Buy milk',
  description: '2 liters, whole',
  completed: false,
  created_at: ISODate('2026-08-16T07:54:35.722Z'),
  updated_at: ISODate('2026-08-16T07:54:35.722Z')
}
```

> **WATCH OUT**
> If every request now hangs for about 30 seconds and then fails with `ServerSelectionTimeoutError`, pymongo cannot reach MongoDB. Check that the container is running (`docker ps`) or that your MongoDB service is started, and that `MONGO_URI` is correct.

5.5 Final project structure

After Steps 1 to 3 your project should look like this. The screenshot shows the same structure in the IDE.

Project tree

```
todo_api/
|-- controllers/
|   |-- __init__.py
|   `-- todo_controller.py
|-- models/
|   |-- __init__.py
|   `-- todo.py
|-- repositories/
|   |-- __init__.py
|   |-- todo_repository.py              (the contract)
|   |-- in_memory_todo_repository.py    (list implementation)
|   `-- mongo_todo_repository.py        (MongoDB implementation)
|-- services/
|   |-- __init__.py
|   `-- todo_service.py
|-- database.py
|-- dependencies.py                      (the one-line swap lives here)
|-- main.py
|-- pyproject.toml
`-- uv.lock
```

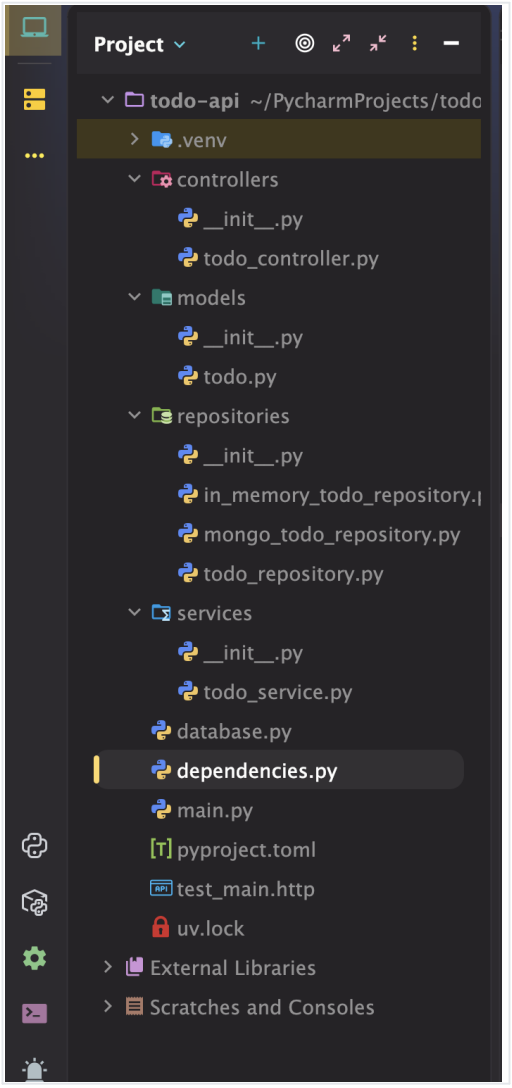


Figure 5.5: The final structure in the IDE.

Notice that `repositories/` now holds **three** files: one contract and two implementations. That is the shape of a well-layered project. Adding a third storage option means adding a file, not editing old ones.

6. Step 4 — Test Your Endpoints

An endpoint you haven't tested isn't finished. It's a guess. We write two kinds of tests here, matching two layers: a **unit test** for the Service (no HTTP, no FastAPI involved at all) and an **integration test** for the Controller (through real HTTP requests, in-process).

What is a test?

A test is a small function that runs part of your code and **asserts** (states) what the result must be. If the assertion is true, the test passes. If not, it fails and tells you exactly which expectation broke. Good tests follow three beats, often called **Arrange, Act, Assert**:

1. **Arrange**: set up what you need (create a service with an empty repository).
2. **Act**: do the one thing being tested (call `create_todo`).
3. **Assert**: check the outcome (`assert todo.title == "Buy milk"`).

	Unit test	Integration test
Tests	One piece in isolation (here, the Service)	Several pieces working together (here, Controller + Service + Repository)
Speaks HTTP?	No	Yes (simulated, in-process)
Speed	Very fast	Fast, a little slower
Catches	Wrong business logic	Wrong routes, status codes, JSON shape, wiring
Our file	`tests/test_todo_service.py`	`tests/test_todo_controller.py`

6.1 Install test dependencies

Terminal

```
uv add --dev pytest httpx
```

Part	Meaning
`pytest`	The most popular Python testing tool. It finds your tests, runs them and reports the results.
`httpx`	An HTTP client library. FastAPI's `TestClient` is built on it, and it is not bundled automatically, so we install it.
`--dev`	Records these as **development-only** dependencies. They are needed to work on the project, not to run it in production.

Add the tests folder:

Terminal (macOS / Linux)

```
mkdir tests
touch tests/__init__.py tests/test_todo_service.py tests/test_todo_controller.py
```

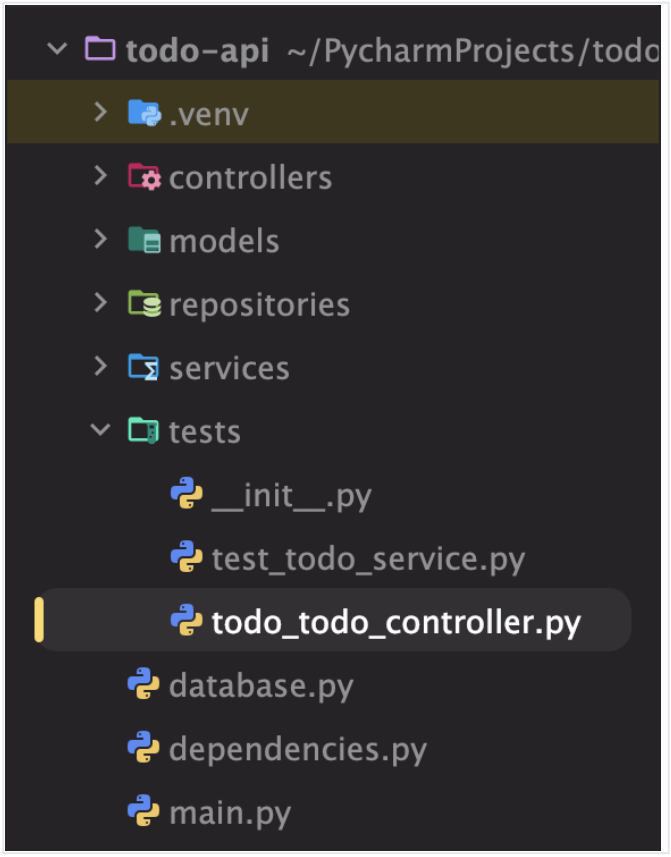


Figure 6.1: The tests folder in the project tree.

> **WATCH OUT**
> **File names matter to pytest.** By default pytest only collects files named `test_*.py` (or `*_test.py`) and functions named `test_*`. In this screenshot the controller test file is named `todo_todo_controller.py`, a typo. With that name pytest would silently **skip** it. Name yours exactly `test_todo_controller.py`.

> **NOTE**
> The empty `tests/__init__.py` also matters. It lets pytest add the project root to Python's import path, which is what makes `from main import app` and `from services.todo_service import ...` work inside the tests.

6.2 Unit test the Service

This file never imports FastAPI. It builds a `TodoService` directly against a throwaway `InMemoryTodoRepository` and checks the business logic in isolation. The same repository swap-ability from Step 3 is what makes this possible: the Service does not care which repository it gets, so a test can hand it a simple one.

tests/test_todo_service.py

```python
from uuid import uuid4
import pytest
from models.todo import TodoCreate
from repositories.in_memory_todo_repository import InMemoryTodoRepository
from services.todo_service import TodoService, TodoNotFoundError


def test_create_todo_returns_a_todo_with_the_given_title():
    service = TodoService(InMemoryTodoRepository())

    todo = service.create_todo(TodoCreate(title="Buy milk"))

    assert todo.title == "Buy milk"
    assert todo.completed is False


def test_get_todo_raises_when_it_does_not_exist():
    service = TodoService(InMemoryTodoRepository())

    with pytest.raises(TodoNotFoundError):
        service.get_todo(uuid4())


def test_delete_todo_removes_it_from_the_repository():
    service = TodoService(InMemoryTodoRepository())
    todo = service.create_todo(TodoCreate(title="Buy milk"))

    service.delete_todo(todo.id)

    with pytest.raises(TodoNotFoundError):
        service.get_todo(todo.id)
```

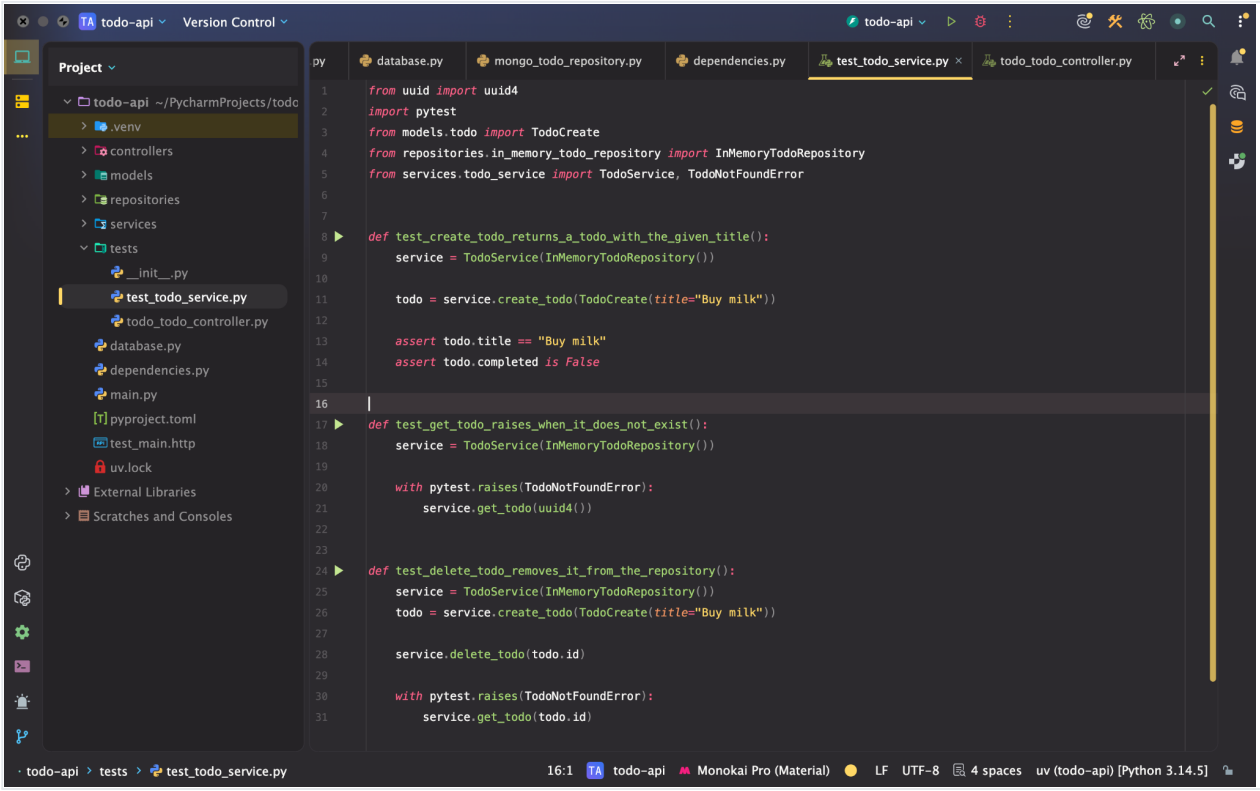


Figure 6.2: tests/test_todo_service.py.

Code	What it does
`TodoService(InMemoryTodoRepository())`	**Arrange.** A brand-new service with a brand-new empty list. Every test builds its own, so tests never affect each other.
`service.create_todo(TodoCreate(title="Buy milk"))`	**Act.** Calls the real Service method with a DTO.
`assert todo.title == "Buy milk"`	**Assert.** If the expression is false, the test fails.
`assert todo.completed is False`	Checks the business default: new todos start incomplete.
`with pytest.raises(TodoNotFoundError):`	Says "the code inside this block **must** raise this error". The test **passes** if it does and **fails** if no error occurs.
`uuid4()`	A random id that cannot exist in an empty repository.

The test names read like sentences: `test_get_todo_raises_when_it_does_not_exist`. When a test fails, its name should tell you what broke without opening the file.

6.3 Integration test the Controller

Here we go through real HTTP requests using FastAPI's `TestClient`. It calls your application **in the same process**, with no server and no network, but all the real routing, validation and status codes run.

The key move is `app.dependency_overrides`. We swap out whatever repository is wired in `dependencies.py` (currently `MongoTodoRepository`) for a fresh `InMemoryTodoRepository`, so tests never touch the real database and never leave data behind.

Fixtures

A **fixture** is a reusable piece of setup. You mark a function with `@pytest.fixture`. Any test that lists the fixture's name as a parameter automatically receives what it provides. In a fixture, everything before `yield` is **setup**, the yielded value is what the test gets, and everything after `yield` is **cleanup** that runs when the test ends.

tests/test_todo_controller.py

```
from uuid import uuid4
import pytest
from fastapi.testclient import TestClient
from main import app
from dependencies import get_todo_repository
from repositories.in_memory_todo_repository import InMemoryTodoRepository


@pytest.fixture
def client():
    repository = InMemoryTodoRepository()
    app.dependency_overrides[get_todo_repository] = lambda: repository
    yield TestClient(app)
    app.dependency_overrides.clear()


def test_create_todo_returns_201_and_the_created_todo(client):
    response = client.post("/todos/", json={"title": "Buy milk"})

    assert response.status_code == 201
```

```
    assert response.json()["title"] == "Buy milk"


def test_get_missing_todo_returns_404(client):
    response = client.get(f"/todos/{uuid4()}")

    assert response.status_code == 404


def test_full_crud_flow(client):
    create = client.post("/todos/", json={"title": "Walk the dog"})
    todo_id = create.json()["id"]

    get_response = client.get(f"/todos/{todo_id}")
    assert get_response.status_code == 200

    patch_response = client.patch(f"/todos/{todo_id}", json={"completed": True})
    assert patch_response.json()["completed"] is True

    delete_response = client.delete(f"/todos/{todo_id}")
    assert delete_response.status_code == 204
```

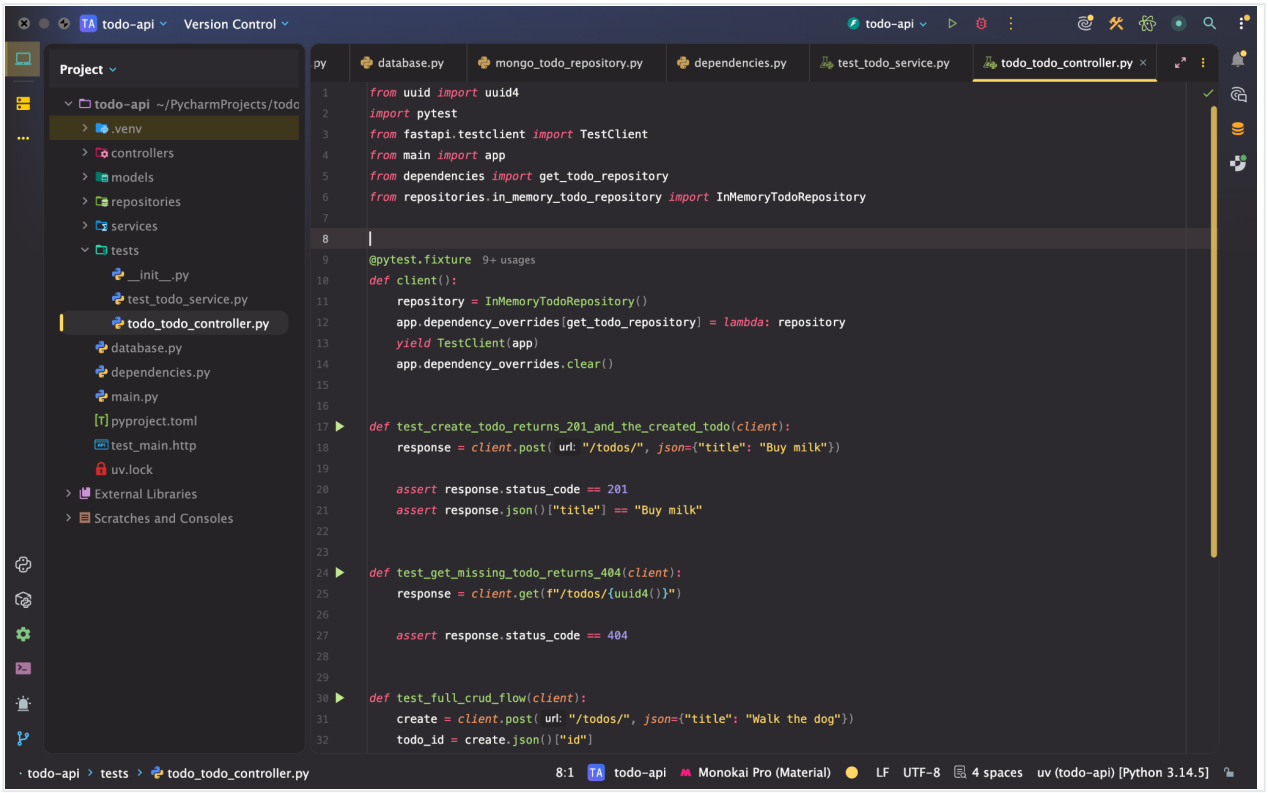


Figure 6.3: tests/test_todo_controller.py (top of the file: the fixture and the first tests).



Figure 6.4: the end of the file: test_full_crud_flow.

The fixture, line by line

Code	What it does
`repository = InMemoryTodoRepository()`	A fresh, empty repository for **this** test.
`app.dependency_overrides[get_todo_repository] = lambda: repository`	A dictionary on the app: "whenever any route asks for `get_todo_repository`, run this function instead". The `lambda` simply returns our fresh repository. The real service is still used, now sitting on the throwaway repository.
`yield TestClient(app)`	Hands a test client to the test. The test runs at this point.
`app.dependency_overrides.clear()`	Cleanup: removes the override so one test cannot leak into the next.

The tests, one by one

Test	What it proves
`test_create_todo_returns_201_and_the_created_todo`	POST works, returns **201**, and the JSON contains the title we sent.
`test_get_missing_todo_returns_404`	The Service's `TodoNotFoundError` really is translated into a **404** by the Controller.
`test_full_crud_flow`	Create, read, patch and delete work **together**. It reads the `id` from the first response and uses it in the later requests.

> NOTE
>
> **Notice:** the fixture creates one `InMemoryTodoRepository` per test and returns that same instance on every call. That is what lets `test_full_crud_flow` create a todo in one request and still find it in the next. A fresh repository per test keeps tests from leaking state into each other.

> **WHY IT MATTERS**
> This is the biggest practical reward of dependency injection. Because the repository arrives through `Depends`, a test can replace it with one line, and no test ever needs MongoDB running. If the route had created its own database connection inside, this would be impossible.

6.4 Run the tests

Terminal

```
uv run pytest -v
```

`-v` means **verbose**: one line per test instead of a row of dots. You should see output similar to this:

Expected output (abridged)

```
tests/test_todo_controller.py::test_create_todo_returns_201_and_the_created_todo PASSED
tests/test_todo_controller.py::test_get_missing_todo_returns_404 PASSED
tests/test_todo_controller.py::test_full_crud_flow PASSED
tests/test_todo_service.py::test_create_todo_returns_a_todo_with_the_given_title PASSED
tests/test_todo_service.py::test_get_todo_raises_when_it_does_not_exist PASSED
tests/test_todo_service.py::test_delete_todo_removes_it_from_the_repository PASSED

============================== 6 passed ==============================
```

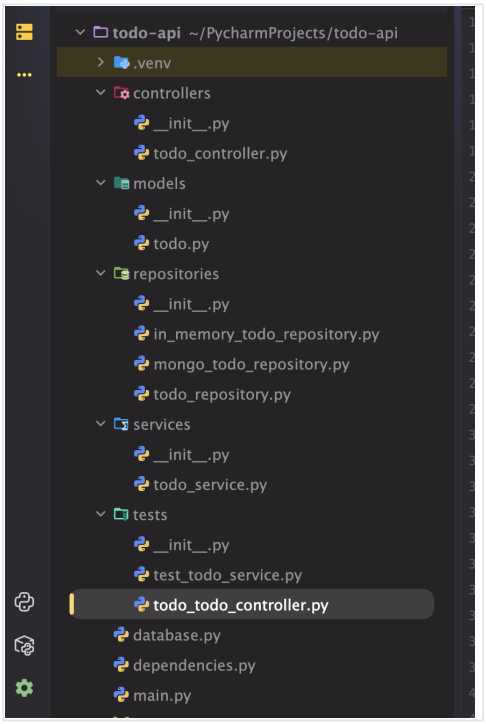


Figure 6.5: The project structure after adding the tests folder.

Useful pytest commands

Command	Runs
`uv run pytest -v`	Every test, with one line each.
`uv run pytest tests/test_todo_service.py`	Only one file.
`uv run pytest -k crud`	Only tests whose name contains "crud".
`uv run pytest -x`	Stops at the first failure.

TRY IT

Break something on purpose. In `TodoService.get_todo`, delete the `raise` line and run the tests again. Watch which tests fail and read the failure message. Then put the line back. Learning to **read** a failing test is as important as writing a passing one.

What would you test next?

- Updating a todo that does not exist returns 404 (both service and controller).
- Creating a todo with an empty title returns 422.
- A `PATCH` with only `completed` leaves the title untouched.
- Listing todos returns every created todo.

7. Putting It All Together

7.1 One request, every layer

Here is `PATCH /todos/{id}` with `{"completed": true}`, traced through the finished project. Every row points to a real line in the code you wrote.

Layer / file	Function	What it contributes
FastAPI	routing and validation	Finds the PATCH route; checks `{id}` is a valid UUID; parses the body into `TodoUpdate`. Otherwise: 422.
`dependencies.py`	`get_todo_repository`, `get_todo_service`	Provide the repository and build the service.
`todo_controller.py`	`update_todo`	Passes `todo_id` and `payload` to the Service; catches `TodoNotFoundError` and turns it into 404.
`todo_service.py`	`update_todo`	`model_dump(exclude_unset=True)` gives `{"completed": True}`; asks the repository; raises if nothing was found.
`*_todo_repository.py`	`update`	Applies the change in the list **or** in MongoDB and returns the updated `Todo`.
`models/todo.py`	`Todo`	The shape of the object returned, and of the JSON that FastAPI sends back.

7.2 Where does this code belong?

When you add a feature, the hardest question is usually "which file?". Use this table.

If the code...	It belongs in the...	Because...
describes the fields of an object or the shape of a request	Model	It is pure data description.
says "titles must be unique" or "max 100 todos"	Service	It is a business rule, true no matter how the app is accessed.
runs a database query or reads a file	Repository	Storage details must stay in one layer.
chooses a status code or builds an `HTTPException`	Controller	It is about HTTP.
decides which repository class to use	`dependencies.py`	It is wiring, not logic.
reads `MONGO_URI` or other configuration	`database.py` / settings	Configuration is separate from behaviour.

7.3 The layering checklist

You can review any file with these questions:

File	Ask yourself	Red flag
Controller	Does the route only call the Service and translate errors?	An `if` about todo data; a database call; importing a repository.
Service	Does it only talk to the repository **contract**?	Importing `fastapi`, `HTTPException`, `pymongo`, or a concrete repository.
Repository	Does it only store and fetch?	A rule such as "no duplicate titles"; any HTTP concept.
Model	Does it only describe data?	Database calls or business decisions.
`dependencies.py`	Is it the only place that names concrete classes?	Concrete repository names appearing in the Service or Controller.

7.4 The patterns you used

You have now used several well-known design patterns and principles. Knowing their names helps you talk about your design, and search for more.

Name	Where in this project
Repository pattern	`TodoRepository` hides how data is stored behind collection-like methods.
DTO	`TodoCreate` and `TodoUpdate` carry only what the client may send.
Dependency injection	`Depends(...)`, and the Service receiving its repository in `__init__`.
Dependency inversion (the D in SOLID)	The Service depends on the abstract `TodoRepository`, not on a concrete class.
Single responsibility (the S in SOLID)	Each layer, and each file, has one reason to change.
Liskov substitution (the L in SOLID)	Any `TodoRepository` can stand in for any other, so the in-memory one in tests and MongoDB in production behave interchangeably.

7.5 Mistakes to avoid

- **Leaking HTTP into the Service** (importing `HTTPException` there). The Service must stay usable without a web framework.
- **Using the entity as the request body.** Use input DTOs, so clients cannot set server-controlled fields.
- **Creating the repository inside a route or per request.** Use the shared, injected one.
- **Letting two implementations of one contract behave differently** (like the `updated_at` gap). Swapping implementations should not change behaviour.

8. Troubleshooting

Most problems in this project are one of the handful below. Read the **last line** of an error message first. It usually names the real cause.

What you see	Likely cause	Fix
`ModuleNotFoundError: No module named 'models'`	You ran the command from the wrong folder, or a folder is missing `__init__.py`.	`cd todo_api` first; add an empty `__init__.py` to every layer folder.
`ModuleNotFoundError: No module named 'fastapi'`	You ran `python` or `uvicorn` directly, outside the project environment.	Run through uv: `uv run uvicorn main:app --reload`.
`Error loading ASGI app. Could not import module "main"`	Wrong folder, or a syntax or import error inside your code.	Run from the project root; read the error printed just above.
`TypeError: Can't instantiate abstract class ...`	A repository class is missing one of the five contract methods.	Implement every method listed in the message.
422 Unprocessable Entity	The request does not match the model: missing title, empty title, bad UUID.	Read `detail[].loc` and `msg` in the response. They name the exact field.
404 on a todo you just created	You restarted the server while using the in-memory list, or the id is wrong.	Create the todo again; or complete Step 3 so data persists.
All todos disappear between requests	The repository is created inside `get_todo_repository()` instead of once at module level.	Create `_repository` once at the top of `dependencies.py`.
Requests hang, then `ServerSelectionTimeoutError`	MongoDB is not running or `MONGO_URI` is wrong.	`docker ps`, `docker start todo-mongo`, or start your MongoDB service.
`RuntimeError` mentioning `httpx` when running tests	The test client needs `httpx`.	`uv add --dev httpx`
pytest says "collected 0 items" or skips a file	File or function names do not start with `test_`.	Rename to `test_*.py` and `test_*` functions.
`ImportError: cannot import name ... (circular import)`	Two modules import each other, usually because a lower layer imports a higher one.	Check the dependency rule from chapter 1 and move the import.

9. Exercises and Self-Check

You only really own an architecture once you have **changed** it. These exercises are ordered from small to ambitious. For each one, before you write code, write down **which layers and files will change**. Often the best answer is "only one".

9.1 Practice exercises

#	Exercise	Hint: where does the change go?
1	**Fix the `updated_at` gap.** Make `InMemoryTodoRepository.update` refresh `updated_at`, so it behaves like the MongoDB version.	Repository only. Add `data["updated_at"] = ...` before `model_copy`.
2	**Add a `priority` field** (an integer from 1 to 5, default 3) to todos.	Model (`Todo`, `TodoCreate`, `TodoUpdate`) and the Service's `create_todo`. Does the Mongo repository need changing?
3	**Reject duplicate titles.** Creating a todo whose title already exists must fail with HTTP 409 Conflict.	Service raises a new `DuplicateTitleError`; Controller translates it. What does the Service need from the repository?
4	**Filter the list:** `GET /todos/?completed=true` returns only completed todos.	A query parameter in the Controller, a filter in the Service. Keep the Repository unchanged if you can.
5	**One handler for all 404s.** Remove the three `try/except` blocks and register a single `@app.exception_handler(TodoNotFoundError)`.	Controller / `main.py`. The Service stays untouched. Re-run your tests.
6	**A JSON-file repository.** Write `JsonFileTodoRepository` that saves todos in a `todos.json` file.	One new file in `repositories/` and one line in `dependencies.py`. Nothing else may change.
7	**Choose the repository from the environment.** Read `REPOSITORY=memory` or `mongo` and return the matching repository.	`dependencies.py` only.
8	**Write the missing tests:** update 404, empty title 422, and a partial update that leaves the title intact.	New tests in `tests/`.
9	**Swap to a SQL database** (SQLite or MySQL with SQLAlchemy) by writing another `TodoRepository`.	A new repository plus its connection setup; the Service and Controller must not change.

9.2 Self-check questions

Try to answer each question aloud before you read the short answer. If you can explain these, you understand the architecture.

Question	Short answer
What is the single job of each of the four layers?	Model: shape of data. Repository: storage. Service: business rules. Controller: HTTP in and out.
What is a DTO, and why are `TodoCreate` and `TodoUpdate` separate from `Todo`?	A data carrier. Separate input DTOs stop clients from setting server-controlled fields and let each use case have its own rules.
What does `BaseModel` give you?	Validation, type conversion, defaults, serialization to dict/JSON, and a schema for the docs.
What does `Field(..., min_length=1)` mean?	The field is required (`...`) and must have at least one character.
Why `default_factory` instead of a plain default?	A plain default is evaluated once at import; a factory runs for every new object (new id, current time).
Why does the Service depend on `TodoRepository`, not `InMemoryTodoRepository`?	So any implementation can be plugged in without changing the Service (dependency inversion).
What does `@abstractmethod` enforce?	A subclass cannot be instantiated unless it implements every abstract method.
Why does the Service raise `TodoNotFoundError` and not `HTTPException`?	The Service must not know about HTTP. The Controller translates the domain error into a 404.
What does `Depends` do?	It tells FastAPI to call a function and inject its result, resolving nested dependencies automatically.
What does `exclude_unset=True` do in `update_todo`?	It includes only the fields the client actually sent, enabling partial updates.
Which single file changed when we moved to MongoDB (besides adding new files)?	`dependencies.py`.
How can a controller test avoid using the real database?	`app.dependency_overrides[get_todo_repository] = ...` with an in-memory repository.

9.3 Be ready to defend your design

When you present your project, expect to be asked to **change** it live: swap a dependency, add a field, add a rule. A good answer has three parts: (1) name the layer where the change belongs, (2) explain why the other layers do not need to change, (3) make the change. Understanding the architecture is as important as writing the code.

10. Glossary

Term	Meaning
ABC (Abstract Base Class)	A class that cannot be instantiated directly and that defines methods subclasses must implement.
API	Application Programming Interface: the set of URLs and rules other programs use to talk to your application.
APIRouter	FastAPI object that groups related routes; attached to the app with `include_router`.
ASGI	The standard interface between Python async web servers (like Uvicorn) and web frameworks (like FastAPI).
BaseModel	The Pydantic base class that gives a class validation, conversion and serialization.
Business logic	The rules of your application that exist regardless of HTTP or storage.
Collection / document (MongoDB)	A collection holds documents, like a table holds rows. A document is a JSON-like record.
Composition root	The single place where concrete implementations are chosen and connected (`dependencies.py`).
Contract	A list of methods a class promises to provide (here, `TodoRepository`). Also called an interface.
Controller	The layer that receives HTTP requests, calls the Service and returns HTTP responses.
CRUD	Create, Read, Update, Delete.
Dependency injection	Giving an object the things it needs instead of letting it create them.
Depends	FastAPI's tool for dependency injection.
DTO	Data Transfer Object: a plain object that carries data between parts of a system.
Entity / domain model	The core object of the application (`Todo`).
Fixture	A pytest function that prepares something (like a test client) for tests that ask for it.
HTTPException	FastAPI error that stops a request and returns an error status and message.
Integration test	A test of several parts working together (for example, over HTTP).
Layer	A part of the application with one job and defined rules about what it may know.
Partial update (PATCH)	Changing only some fields of a resource.
Pydantic	The Python data-validation library behind FastAPI's models.
pymongo	MongoDB's official Python driver.

Term	Meaning
pytest	A Python testing framework.
Repository	The layer (and pattern) that hides how data is stored behind collection-like methods.
response_model	The shape FastAPI validates and documents for a route's response.
Service	The layer that holds business rules and coordinates repositories.
Status code	A three-digit number in an HTTP response saying what happened (200, 201, 204, 404, 422…).
Swagger UI	The interactive documentation page FastAPI serves at `/docs`.
Unit test	A test of one small piece in isolation.
uv	The tool we use to create the project, manage the environment and run commands.
UUID	A 128-bit random identifier, such as `7c9e6679-7425-40de-944b-e07fc1f90ae7`.
Uvicorn	The server that runs a FastAPI application.
Validation	Checking that data has the right type and satisfies the rules before using it.
Virtual environment	A private folder of Python packages for one project (`.venv/`).

Closing Note

PARTS 1–3 COMPLETE

You have now built a fully working Todo API: Model, Repository, Service and Controller wired together and doing real CRUD (Part 1); swapped the storage layer to MongoDB with a one-line change (Part 2); and added a test suite that exercises both the business logic and the live endpoints without ever touching the real database (Part 3).

Coming next, in Part 4:

- **Authentication**: adding a User model and login flow, then scoping every todo to the signed-in user so people only ever see their own list.

Remember: the code is the easy part to copy. What makes you an engineer is being able to explain **why each piece lives where it does**, and what you would change, and what you would leave alone, when the requirements move.

Sikiru Ayinlade
Software Engineer / Facilitator
Semicolon